

03_data-overview

June 19, 2026

```
[1]: # %%
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from consts import *
import numpy as np
from itertools import combinations

plt.rcParams.update({"font.size":9})
plt.rcParams.update({"figure.figsize":(16/2.54, 9/2.54)})
sns.set_style("ticks")
sns.set_context("paper")
```

```
[3]: # %% [markdown]
# # Variable Overview
```

```
[4]: # %% [markdown]
# ### Exlcusion Criteria
```

```
[20]: # %%
df_p = pd.read_csv("processed_data/2026-06-19_data_processed_participant.csv")
print("Full Size of participant data: ", df_p.wave.value_counts().to_dict())

#print("size of original data: ", len(df_p_full), " including wave 1 and 2: ",
#      df_p_full["wave"].value_counts().to_dict())
#df_p = df_p_full.loc[(~df_p_full.excl_double) & (~df_p_full.excl_NA) &
#                    (~df_p_full.excl_time)].copy()
#print(f"Nr of distinct participants: {len(df_p['id'].unique())}")
#print(f"    after excluding (i) participants with NA opinions ({sum(df_p_full.
#      excl_NA})) and (ii) participants with less than 1/3 of the median completion
#      time ({sum(df_p_full.excl_time)}) and (iii) double entries (already excluded
#      in preprocessing step)")
#print("size of updated data: ", len(df_p), " including wave 1 and 2: ",
#      df_p["wave"].value_counts().to_dict())
# ids_w1 = df_p.loc[df_p.wave==1, "id"].unique()
# ids_w2 = df_p.loc[df_p.wave==2, "id"].unique()
```

```

inds_bothwaves = df_p["id"].value_counts().reset_index().
    ↪query("count==2")["id"].tolist()
# print("Nr of participants who completed both waves: ", len(inds_bothwaves))
df_p_bothwaves = pd.read_csv("processed_data/
    ↪2026-06-19_data_processed_participant_pivot.csv").query(f"id in_
    ↪{inds_bothwaves}")
print("Full Size of across-wave data: ", len(df_p_bothwaves))
# print(df_p.shape, df_p.columns[50:60])

```

Full Size of participant data: {1: 1170, 2: 961}

Full Size of across-wave data: 903

```

[21]: # %%
df_diff = pd.read_csv("processed_data/2026-06-19_data_processed_differences.
    ↪csv")
# df_diff = df_diff.loc[((df_diff["id"].isin(ids_w1)) & (df_diff["wave"]==1)) |_
    ↪((df_diff["id"].isin(ids_w2)) & (df_diff["wave"]==2))]
print("Full Size of pairwise data (in wave 1 and wave 2): ", df_diff["wave"].
    ↪value_counts().to_dict())
print(df_diff.shape)
print(len(df_diff.loc[df_diff.wave==1,"id"].unique()))
print(len(df_diff.loc[df_diff.wave==2,"id"].unique()))
# print(df_diff.columns)

```

Full Size of pairwise data (in wave 1 and wave 2): {1: 150450, 2: 125463}

(275913, 34)

1170

961

```

[22]: # %% [markdown]
# ## Data

```

```

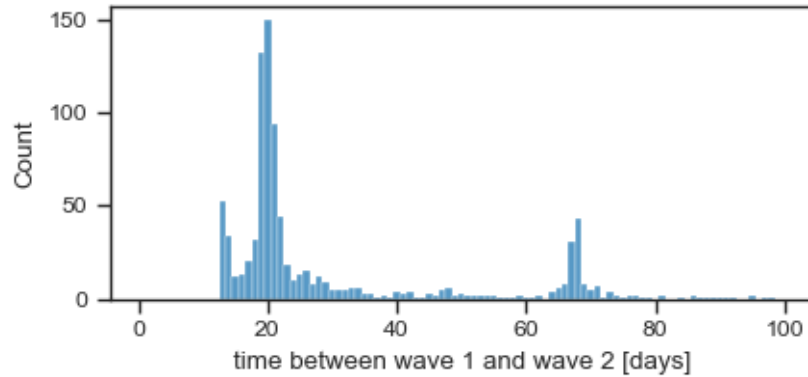
[23]: # %%
plt.figure(figsize=(12/2.54, 5/2.54))
sns.histplot((pd.to_datetime(df_p_bothwaves["wave2_t_completed"]) - pd.
    ↪to_datetime(df_p_bothwaves['wave1_t_completed'])).dt.days, bins=np.arange(0.
    ↪5,100.5))
plt.xlabel("time between wave 1 and wave 2 [days]" )

```

```

[23]: Text(0.5, 0, 'time between wave 1 and wave 2 [days]')

```



```
[24]: # %%
fig, ax = plt.subplots(1,1, sharex=False)
t = 'time_total'
vmax = np.percentile(df_p.assign(t_div_60=df_p[t] / 60)['t_div_60'].values, 95)
    ↳ 3
sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave", bins=
    ↳ np.arange(0,vmax,3), alpha=0.5)
ax.set_title(t)
ax.text(0.95,0.95,
        "\n".join([f"{np.sum(df_p.loc[df_p.wave==w, t].values / 60 < vmax)/df_p.
    ↳ loc[df_p.wave==w, t].count()*100:.1f}% with <{vmax:.1f}min (wave {w})" for w
    ↳ in [1,2]]),
        ha="right", va="top", transform=ax.transAxes, fontsize=7)
ax.set_xlabel("time in min")
fig.tight_layout()

print(f"median ({t}): {df_p[t].div(60).median():.2f} (25%-perc: {df_p[t].
    ↳ div(60).describe()['25%']:.2f}, 75%-perc: {df_p[t].div(60).describe()['75%']:
    ↳ .2f})")

times = ['time_trainingGame', 'time_training', 'time_spam', 'time_spam18dots',
    ↳ 'time_pairwise', 'time_pairwise18pairs']
fig, axs = plt.subplots(2,3, sharex=False)
for ax, t in zip(axs.flatten(), times):
    vmax = np.percentile(df_p[t].dropna().values, 95) * 3 / 60
    sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",
    ↳ bins = np.arange(0,vmax,0.5), alpha=0.5, ax=ax, legend=False)
    ax.set_title(t)
    ax.text(0.95,0.95,
            "\n".join([f"{np.sum(df_p.loc[df_p.wave==w, t].values / 60 < vmax)/
    ↳ df_p.loc[df_p.wave==w, t].count()*100:.1f}% with <{vmax:.1f}min (wave {w})"
    ↳ for w in [1,2]]),
```

```

        ha="right", va="top", transform=ax.transAxes, fontsize=7)
    ax.set_xlabel("time in min")
fig.tight_layout()

display(df_p[times].div(60).describe())
display(df_p[times].div(60).describe())

# for t in [4,6]:
#     descr = df_p[time_cols(1)[t]].div(60).describe(percentiles=[0.05,0.25,0.
# ↪5,0.75,0.95])
#     print(f"median ({time_cols(1)[t]}): {descr["50%"]:.2f} (25%-perc:
# ↪{descr["25%"]:.2f}, 75%-perc: {descr["75%"]:.2f}), (5%-perc: {descr["5%"]:.
# ↪2f}, 95%-perc: {descr["95%"]:.2f})")
#     descr = df_p[time_cols(2)[t]].div(60).describe(percentiles=[0.05,0.25,0.
# ↪5,0.75,0.95])
#     print(f"median ({time_cols(2)[t]}): {descr["50%"]:.2f} (25%-perc:
# ↪{descr["25%"]:.2f}, 75%-perc: {descr["75%"]:.2f}) (5%-perc: {descr["5%"]:.
# ↪2f}, 95%-perc: {descr["95%"]:.2f})")

```

```

/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/3367252046.py:4
: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
    vmax = np.percentile(df_p.assign(t_div_60=df_p[t] / 60)['t_div_60'].values,
95) * 3

```

```

/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/3367252046.py:5
: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
    sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",
bins = np.arange(0,vmax,3), alpha=0.5)

```

```

/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/3367252046.py:1
9: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
    sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",
bins = np.arange(0,vmax,0.5), alpha=0.5, ax=ax, legend=False)
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/3367252046.py:1
9: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
    sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",

```

```

bins = np.arange(0, vmax, 0.5), alpha=0.5, ax=ax, legend=False)
/var/folders/07/c88jy3fd1lbdwnzsr8gy7w0w0000gp/T/ipykernel_82816/3367252046.py:1
9: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",
bins = np.arange(0, vmax, 0.5), alpha=0.5, ax=ax, legend=False)
/var/folders/07/c88jy3fd1lbdwnzsr8gy7w0w0000gp/T/ipykernel_82816/3367252046.py:1
9: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",
bins = np.arange(0, vmax, 0.5), alpha=0.5, ax=ax, legend=False)
median (time_total): 20.60 (25%-perc: 15.08, 75%-perc: 30.74)
/var/folders/07/c88jy3fd1lbdwnzsr8gy7w0w0000gp/T/ipykernel_82816/3367252046.py:1
9: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
sns.histplot(df_p.assign(t_div_60=df_p[t] / 60), x='t_div_60', hue="wave",
bins = np.arange(0, vmax, 0.5), alpha=0.5, ax=ax, legend=False)

```

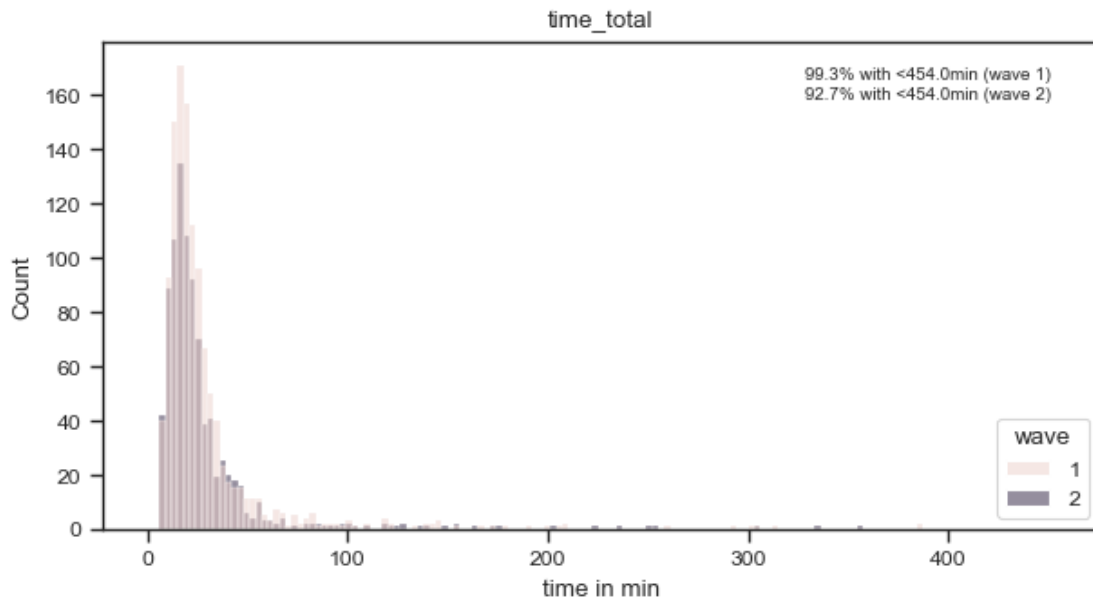
	time_trainingGame	time_training	time_spam	time_spam18dots \
count	2131.000000	2131.000000	2131.000000	1831.000000
mean	2.263890	101.814907	52.473025	1.982186
std	58.441322	1527.680292	1167.174120	32.867415
min	0.083333	0.300000	0.283333	-536.233333
25%	0.366667	1.433333	1.083333	0.816667
50%	0.533333	2.200000	1.600000	1.233333
75%	0.800000	3.516667	2.416667	1.883333
max	2631.750000	44530.183333	46892.716667	1298.333333

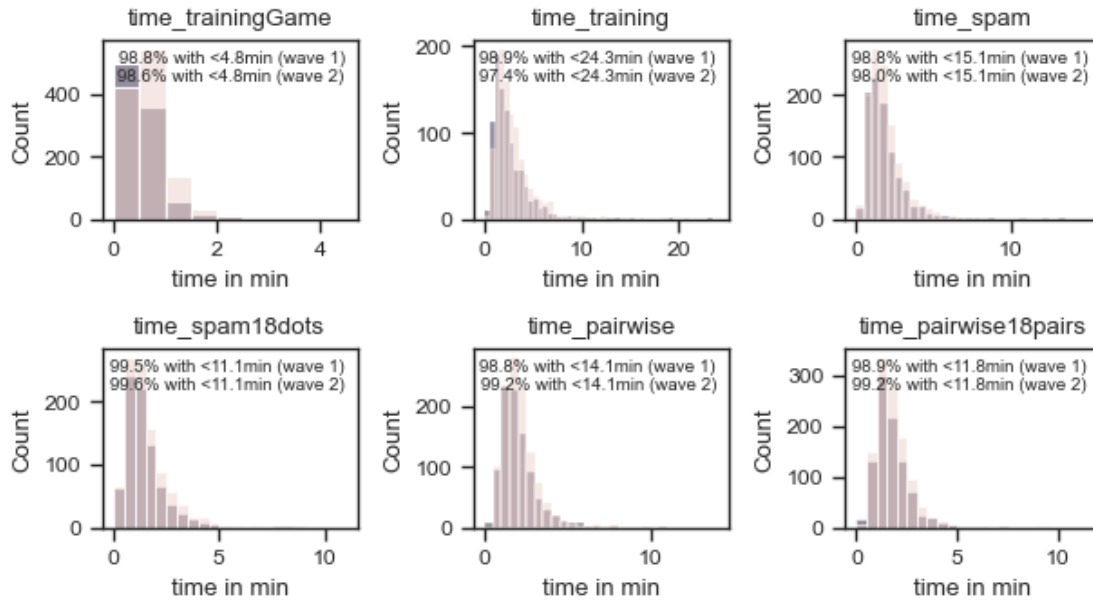
	time_pairwise	time_pairwise18pairs
count	2131.000000	2131.000000
mean	3.152636	2.682739
std	29.740875	29.302688
min	0.350000	0.283333
25%	1.366667	1.200000

50%	1.866667	1.600000
75%	2.533333	2.183333
max	1348.666667	1348.550000

	time_trainingGame	time_training	time_spam	time_spam18dots \
count	2131.000000	2131.000000	2131.000000	1831.000000
mean	2.263890	101.814907	52.473025	1.982186
std	58.441322	1527.680292	1167.174120	32.867415
min	0.083333	0.300000	0.283333	-536.233333
25%	0.366667	1.433333	1.083333	0.816667
50%	0.533333	2.200000	1.600000	1.233333
75%	0.800000	3.516667	2.416667	1.883333
max	2631.750000	44530.183333	46892.716667	1298.333333

	time_pairwise	time_pairwise18pairs
count	2131.000000	2131.000000
mean	3.152636	2.682739
std	29.740875	29.302688
min	0.350000	0.283333
25%	1.366667	1.200000
50%	1.866667	1.600000
75%	2.533333	2.183333
max	1348.666667	1348.550000



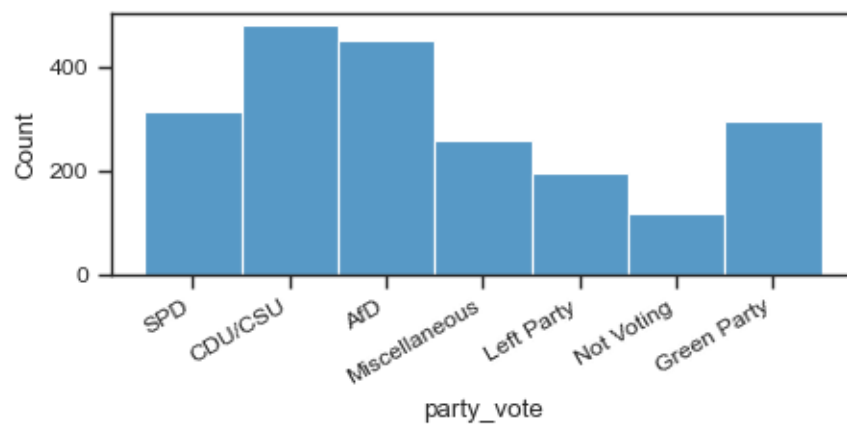
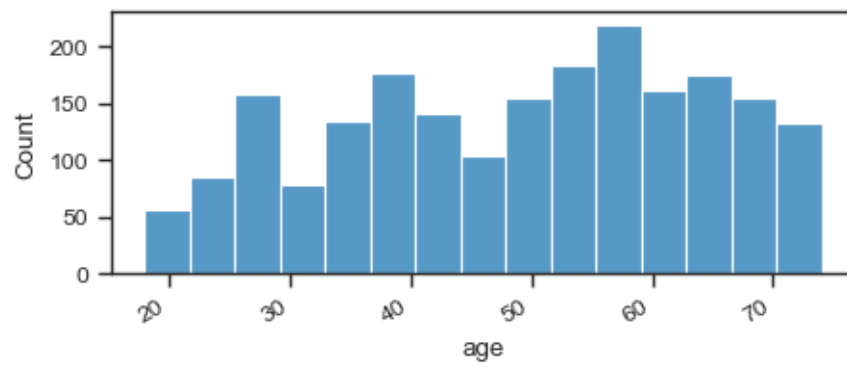
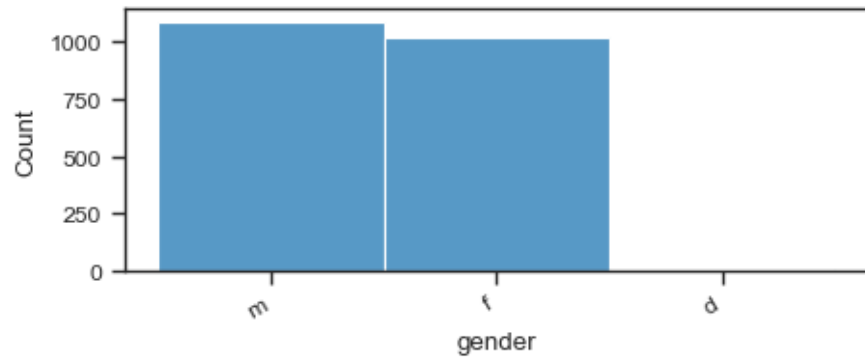


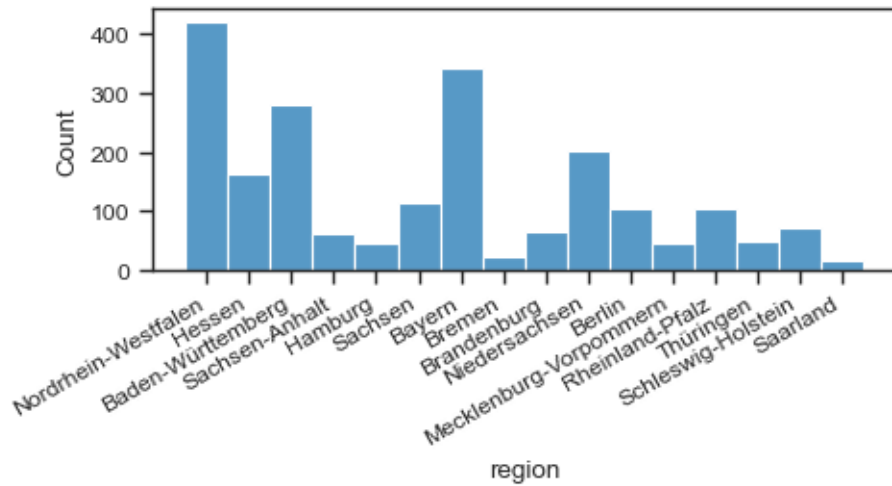
```
[25]: # %% [markdown]
# Note: some of the times are negative and should not be negative. This is
      ↪ probably because they reloaded the page and this changed the visited time.
```

```
[26]: # %% [markdown]
# ## Demographics
#
```

```
[27]: # %% [markdown]
# ### Age Gender Region (bilendi meta-data)
```

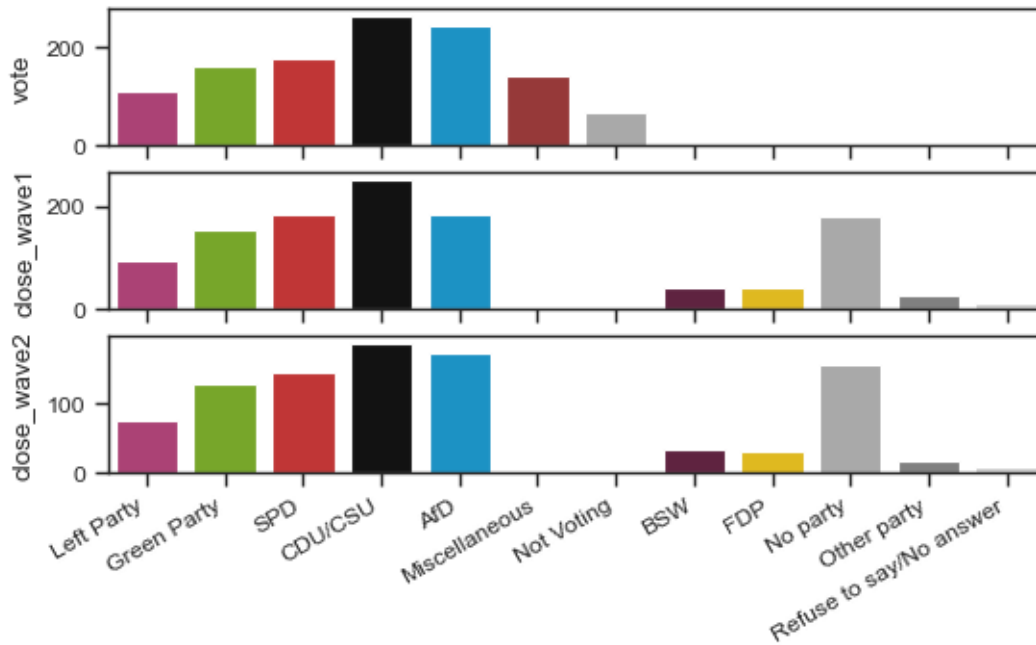
```
[28]: # %%
demo_cols = ["gender", "age", "party_vote", "region"] # participant-level
      ↪ constants
for d in demo_cols:
    fig = plt.figure(figsize=(5,2))
    ax = plt.axes()
    sns.histplot(df_p[d], ax=ax)
    fig.autofmt_xdate()
```





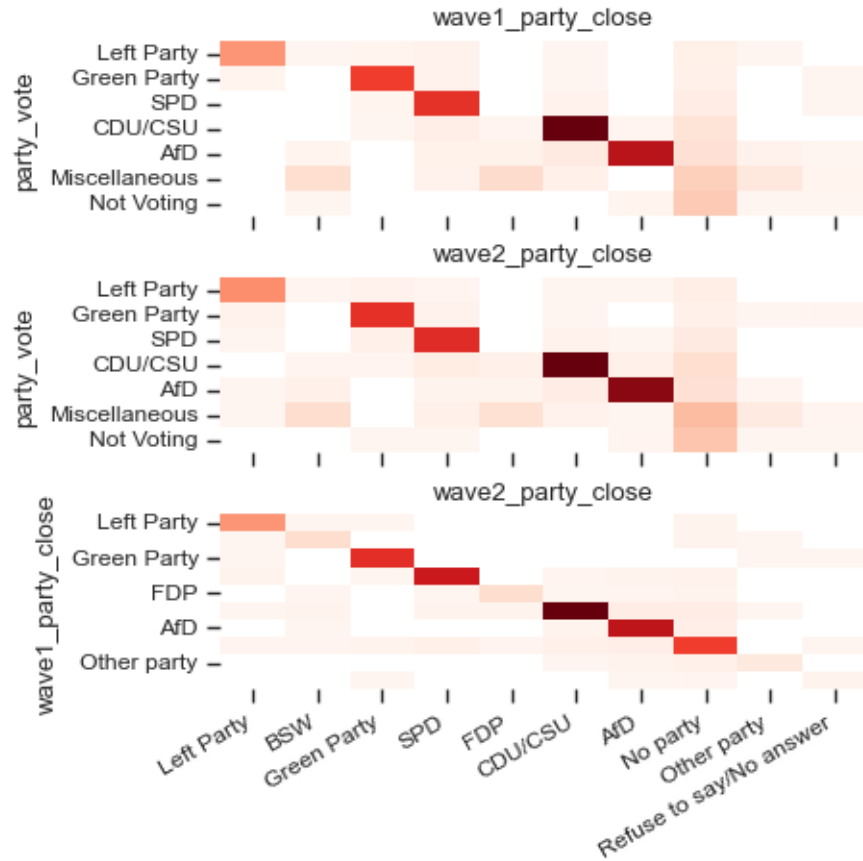
```
[29]: # %% [markdown]
# #### Parties / political identity
```

```
[30]: # %%
party_cmap["Miscellaneous"] = "brown"
party_cmap["Not Voting"] = "darkgrey"
fig, axs = plt.subplots(3,1, sharex=True)
for d, w, name, ax in zip(["party_vote", "party_close", "party_close"],
    ↳ [1,1,2], ["vote", "close_wave1", "close_wave2"], axs.flatten()):
    sns.barplot(df_p.loc[df_p.wave==w, d].value_counts()[parties_vote if "vote"
    ↳ in d else parties_full].reset_index(), ax=ax, y="count", x=d, hue=d,
    ↳ palette=party_cmap)
    fig.autofmt_xdate()
    ax.set_ylabel(name)
    ax.set_xlabel("")
```



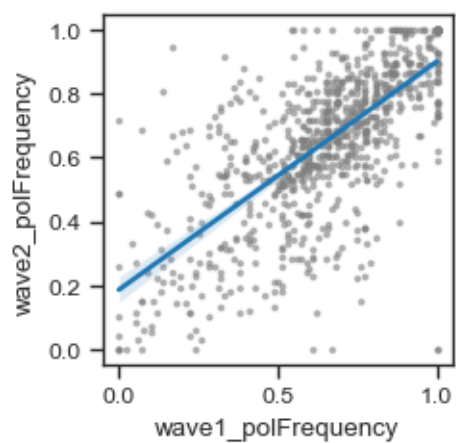
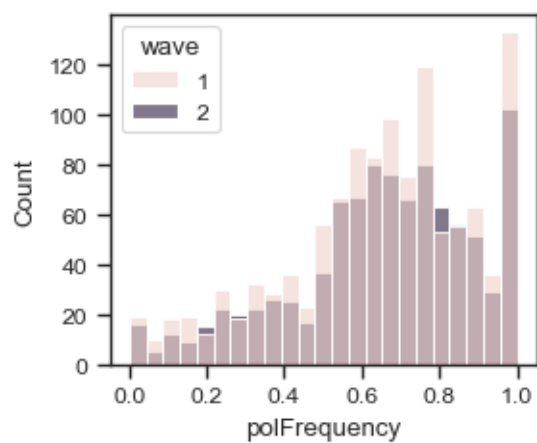
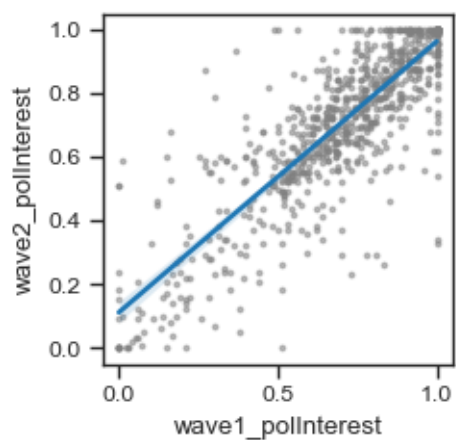
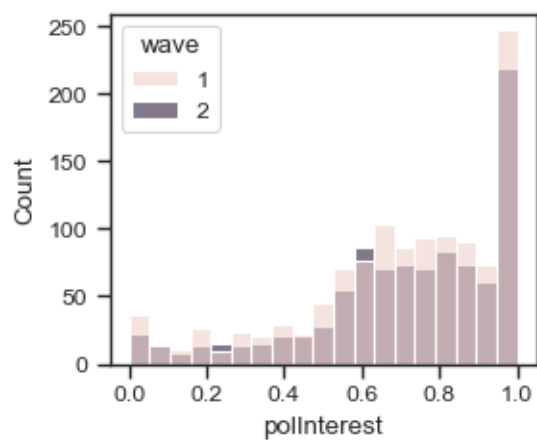
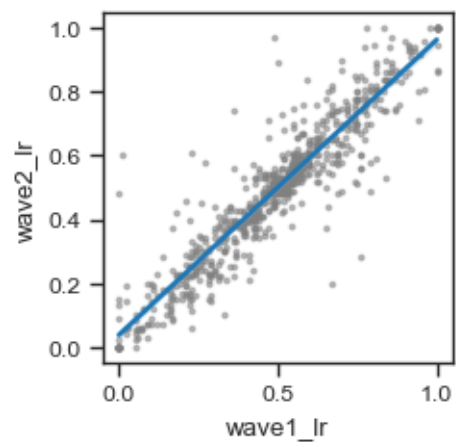
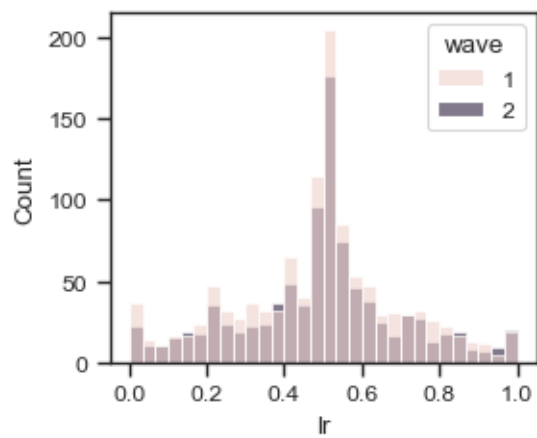
```
[31]: # %%
fig, axs = plt.subplots(3,1, figsize=(12/2.54, 12/2.54), sharex=True,
    ↪sharey=False)
for (d1, d2), ax in zip(combinations(["party_vote", "wave1_party_close",
    ↪"wave2_party_close"], 2), axs):

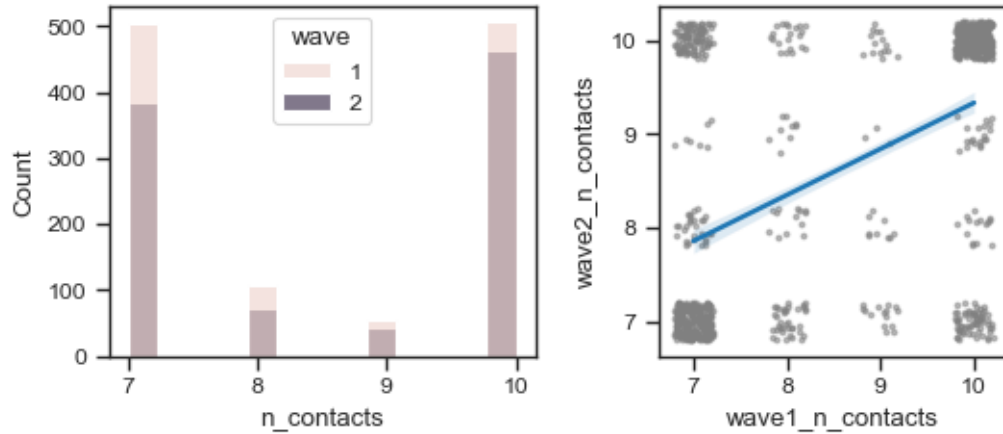
    sns.heatmap(df_p_bothwaves[[d1,d2,"age"]].pivot_table(index=d1,columns=d2,
    ↪aggfunc="count",values="age").loc[parties_full if "wave" in d1 else
    ↪parties_vote, parties_full if "wave" in d2 else parties_vote], cmap="Reds",
    ↪ax=ax, cbar=False,)
    # ax.set_aspect("equal")
    ax.set_title(d2)
    # ax.set_ylabel("")
axs[-1].set_xlabel("")
fig.autofmt_xdate(rotation=30)
fig.tight_layout()
```



```
[32]: # %%
for var in ["lr", "polInterest", "polFrequency", "n_contacts"]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6)
    sns.regplot(df_p_bothwaves[["wave2_"+var, "wave1_"+var]], x="wave1_"+var,
    ↪y="wave2_"+var, ax=axs[1], scatter_kws={"s":3, "alpha":0.5, "color":"grey"},
    ↪y_jitter=0.2*(var=="n_contacts"), x_jitter=0.2*(var=="n_contacts"))
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
```

```
correlation wave 2 wave 1 lr: 0.9285788569852925
correlation wave 2 wave 1 polInterest: 0.8563886946865267
correlation wave 2 wave 1 polFrequency: 0.7105563299702032
correlation wave 2 wave 1 n_contacts: 0.48829717501188447
```





```
[33]: # %%
for var in ["P_tot"]+[f"P_{q}" for q in questions_sc]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6)
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],
    ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
descr1P = df_p[["P_tot"]+[f"P_{q}" for q in questions_sc]].describe()
display(descr1P)
```

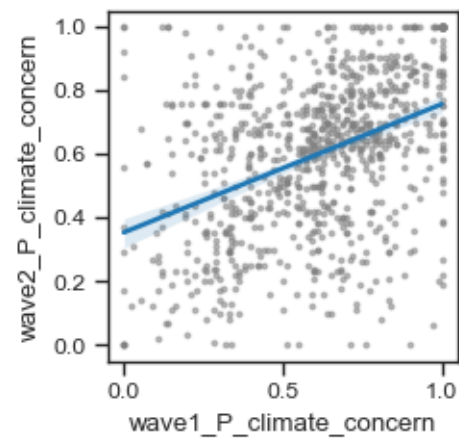
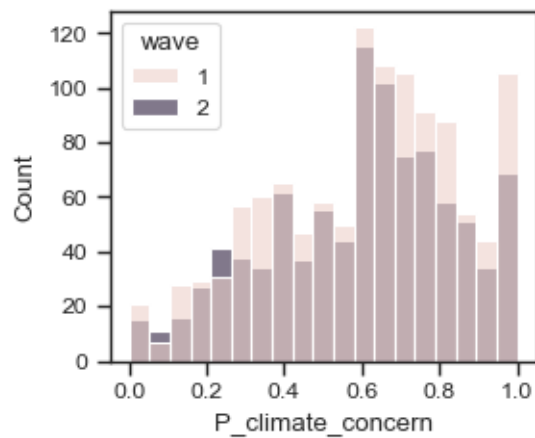
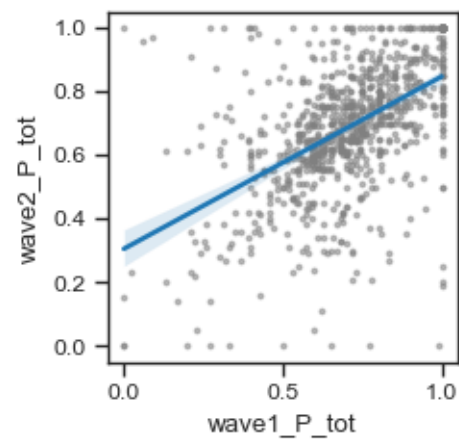
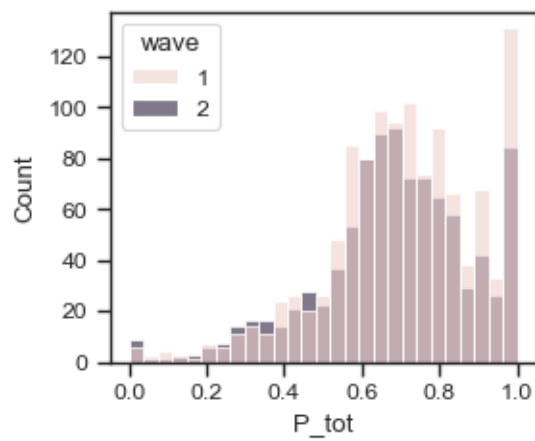
```
correlation wave 2 wave 1 P_tot: 0.5197140603942746
correlation wave 2 wave 1 P_climate_concern: 0.406655671502216
correlation wave 2 wave 1 P_gay_marriage: 0.40457221775802465
correlation wave 2 wave 1 P_rights_indep_integration: 0.359740983017458
correlation wave 2 wave 1 P_econ_inequality: 0.35208645451441983
correlation wave 2 wave 1 P_regulate_internet: 0.32835752880320485
correlation wave 2 wave 1 P_east_germans: 0.3575416282293902
```

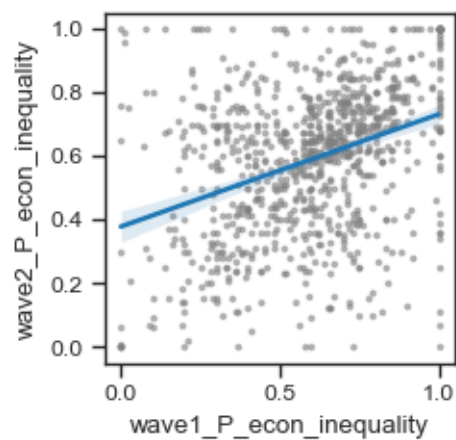
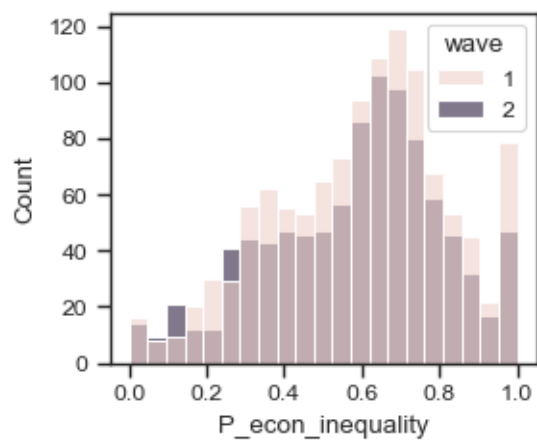
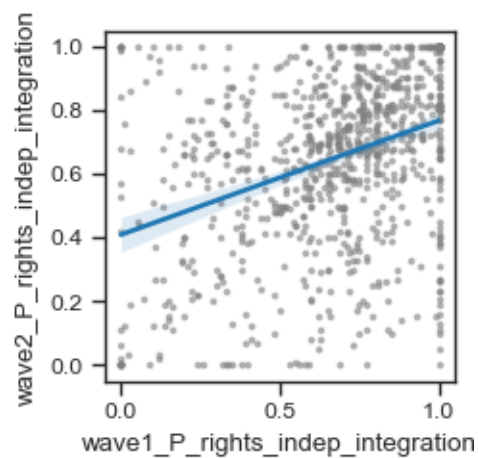
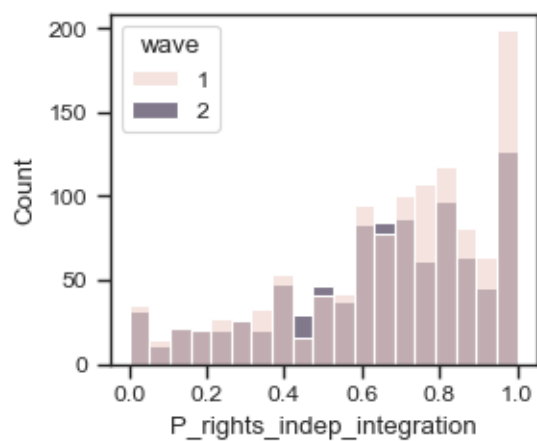
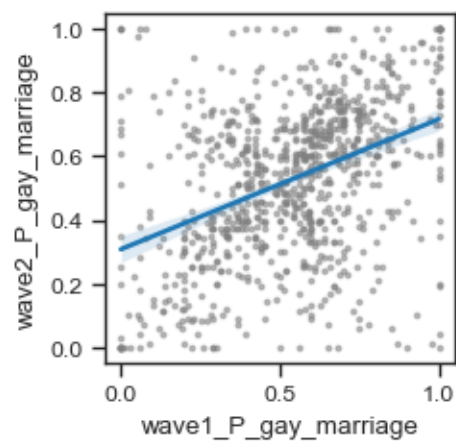
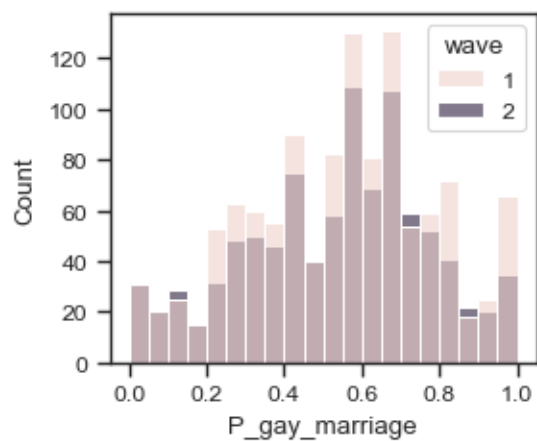
	P_tot	P_climate_concern	P_gay_marriage \
count	2131.000000	2131.000000	2131.000000
mean	0.694904	0.601694	0.538888
std	0.196952	0.242884	0.241068
min	0.000000	0.000000	0.000000
25%	0.590000	0.420000	0.370000
50%	0.700000	0.630000	0.570000
75%	0.830000	0.770000	0.700000
max	1.000000	1.000000	1.000000

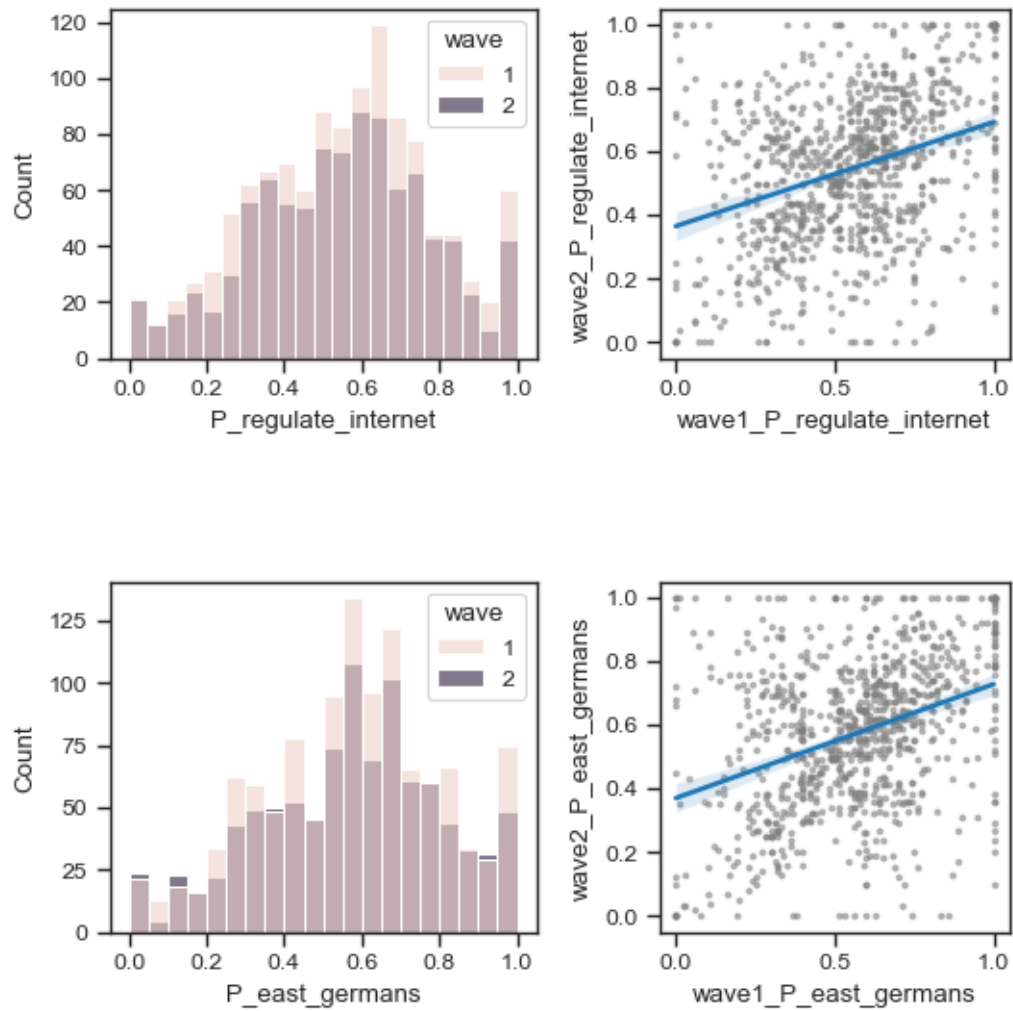
	P_rights_indep_integration	P_econ_inequality	P_regulate_internet \
count	2131.000000	2131.000000	2131.000000

mean	0.660699	0.591764	0.546391
std	0.264068	0.227786	0.230033
min	0.000000	0.000000	0.000000
25%	0.510000	0.430000	0.380000
50%	0.710000	0.620000	0.570000
75%	0.860000	0.750000	0.700000
max	1.000000	1.000000	1.000000

	P_east_germans
count	2131.000000
mean	0.570601
std	0.233402
min	0.000000
25%	0.400000
50%	0.590000
75%	0.730000
max	1.000000

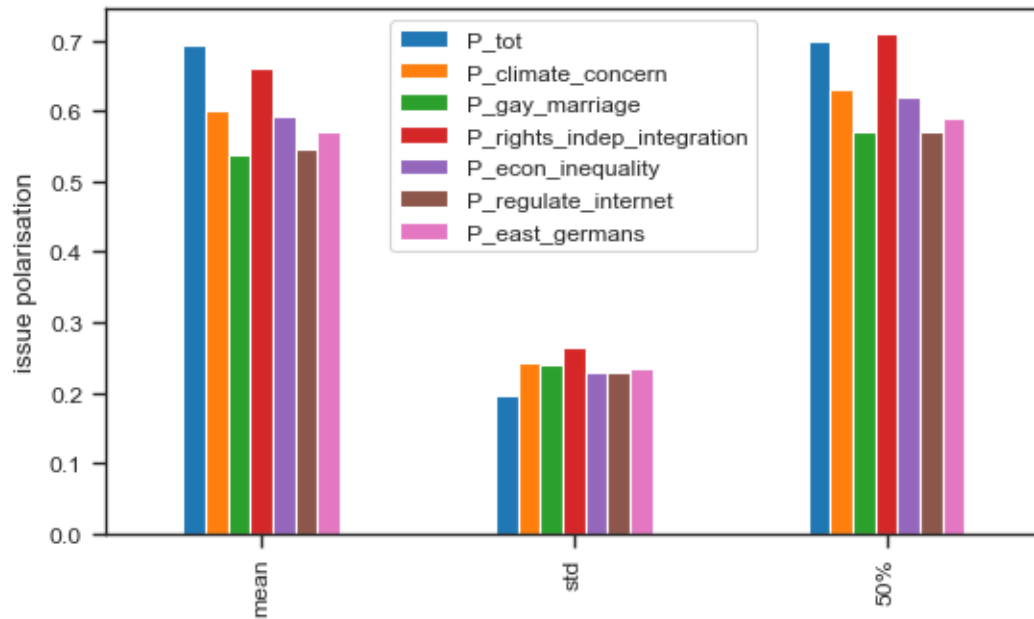






```
[34]: # %%
      descr1P.loc[["mean", "std", "50%"]].plot.bar()
      plt.ylabel("issue polarisation")
```

```
[34]: Text(0, 0.5, 'issue polarisation')
```

```
[35]: # %% [markdown]
      # ## Issue Importance
```

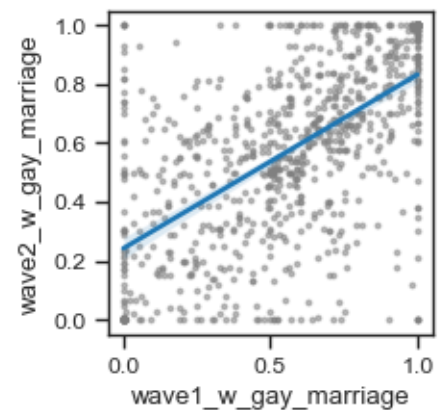
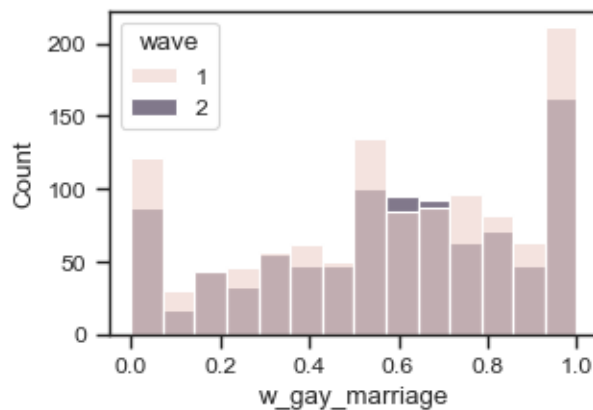
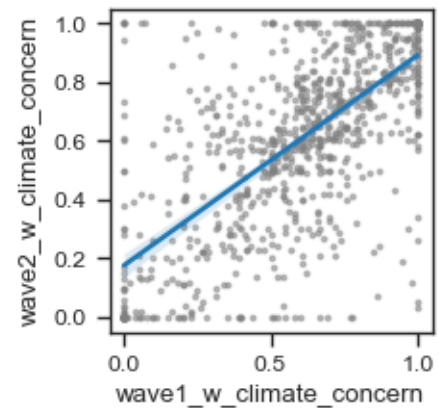
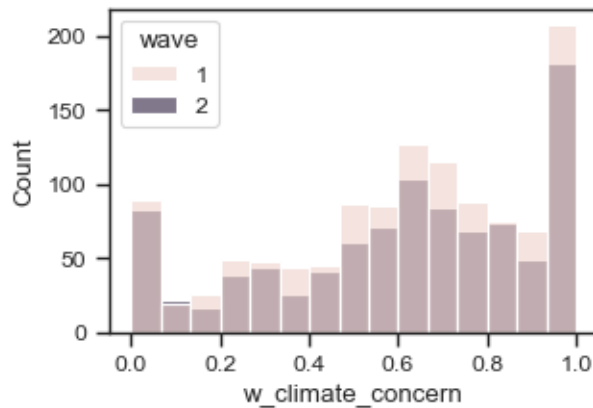
```
[36]: # %%
for var in [f"w_{q}" for q in questions_sc]:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6)
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],
    ↪ scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪ 'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()
descr1w = df_p[[f"w_{q}" for q in questions_sc]].describe()
display(descr1w)
```

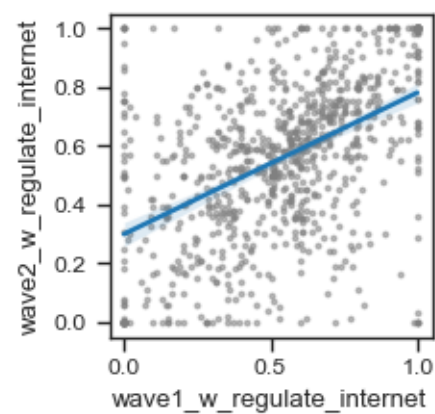
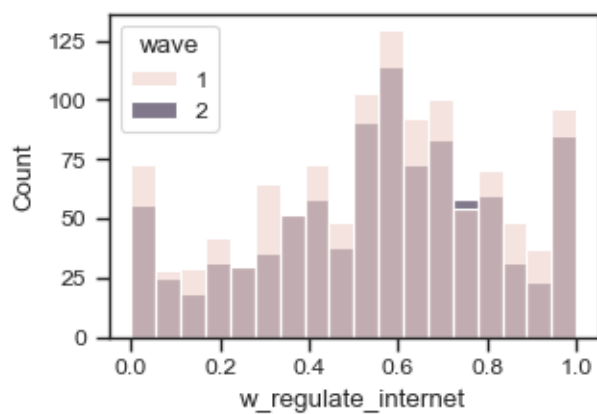
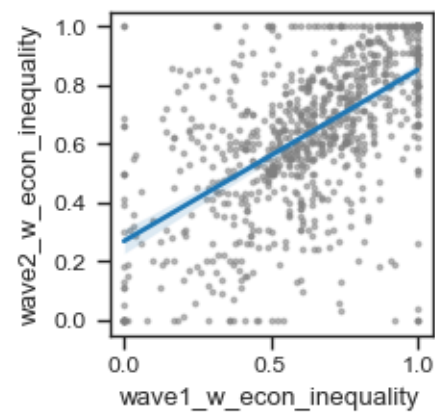
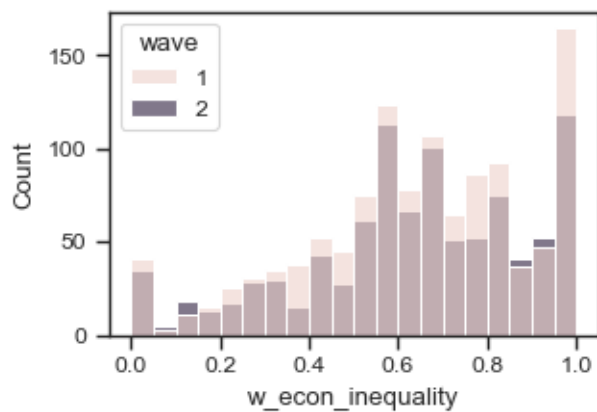
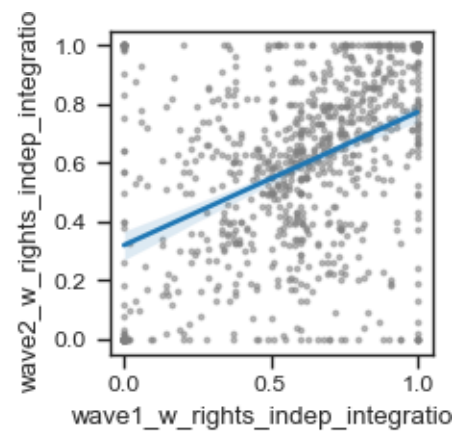
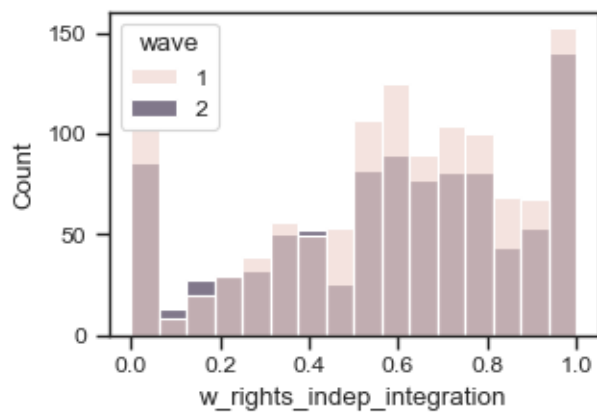
```
correlation wave 2 wave 1 w_climate_concern: 0.6900490845651501
correlation wave 2 wave 1 w_gay_marriage: 0.6066359275019365
correlation wave 2 wave 1 w_rights_indep_integration: 0.434059445041478
correlation wave 2 wave 1 w_econ_inequality: 0.5853821243834351
correlation wave 2 wave 1 w_regulate_internet: 0.48399610550840083
correlation wave 2 wave 1 w_east_germans: 0.6578730519621234
```

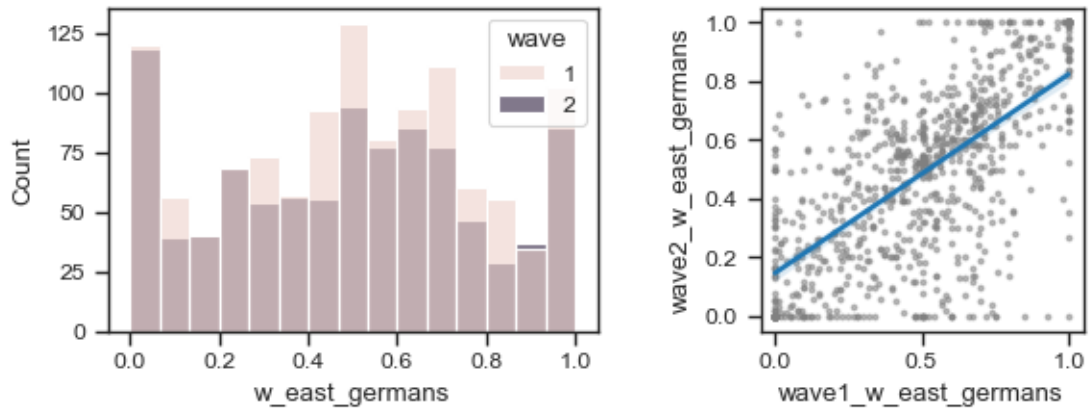
	w_climate_concern	w_gay_marriage	w_rights_indep_integration \
count	2131.000000	2131.000000	2131.000000
mean	0.610000	0.579620	0.590465
std	0.299637	0.308808	0.292577

min	0.000000	0.000000	0.000000
25%	0.410000	0.355000	0.390000
50%	0.650000	0.610000	0.620000
75%	0.845000	0.830000	0.810000
max	1.000000	1.000000	1.000000

	w_econ_inequality	w_regulate_internet	w_east_germans
count	2131.000000	2131.000000	2131.000000
mean	0.636283	0.549587	0.488381
std	0.255775	0.269327	0.293636
min	0.000000	0.000000	0.000000
25%	0.500000	0.370000	0.260000
50%	0.650000	0.580000	0.500000
75%	0.830000	0.740000	0.700000
max	1.000000	1.000000	1.000000

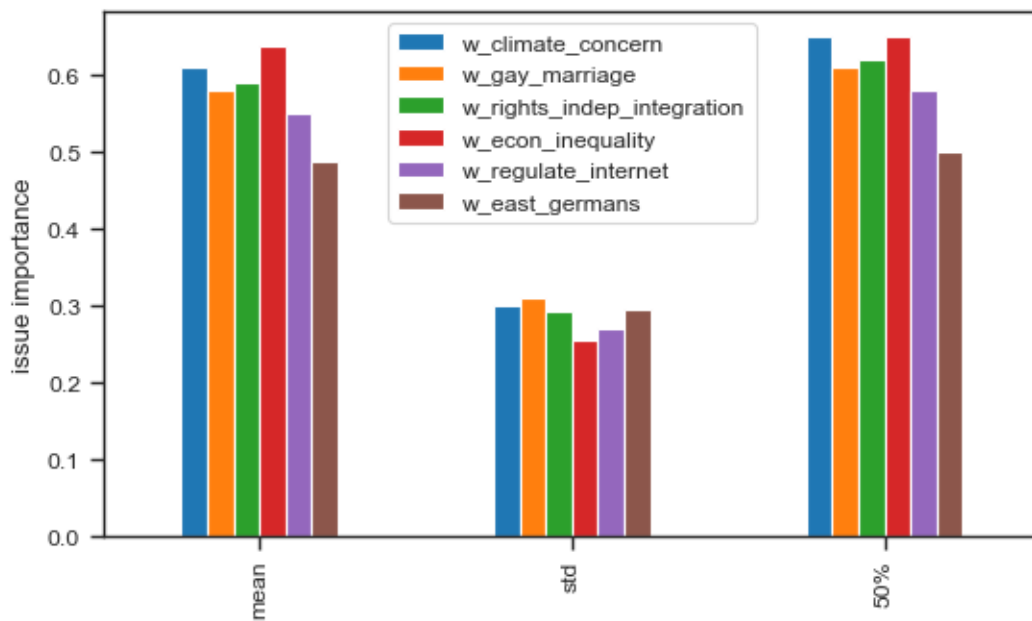






```
[37]: # %%
      descriw.loc[["mean", "std", "50%"]].plot.bar()
      plt.ylabel("issue importance")
```

```
[37]: Text(0, 0.5, 'issue importance')
```



```
[38]: # %%
      df_p[[f"w_{q}" for q in questions_sc]].describe()
```

```
[38]:          w_climate_concern  w_gay_marriage  w_rights_indep_integration  \
count          2131.000000          2131.000000          2131.000000
```

mean	0.610000	0.579620	0.590465
std	0.299637	0.308808	0.292577
min	0.000000	0.000000	0.000000
25%	0.410000	0.355000	0.390000
50%	0.650000	0.610000	0.620000
75%	0.845000	0.830000	0.810000
max	1.000000	1.000000	1.000000

	w_econ_inequality	w_regulate_internet	w_east_germans
count	2131.000000	2131.000000	2131.000000
mean	0.636283	0.549587	0.488381
std	0.255775	0.269327	0.293636
min	0.000000	0.000000	0.000000
25%	0.500000	0.370000	0.260000
50%	0.650000	0.580000	0.500000
75%	0.830000	0.740000	0.700000
max	1.000000	1.000000	1.000000

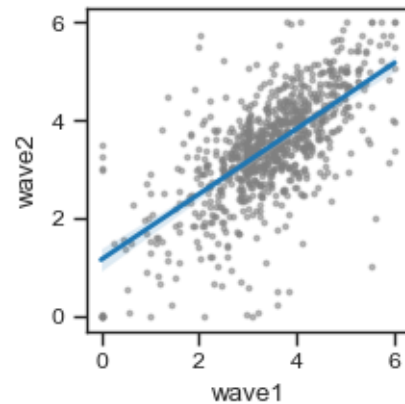
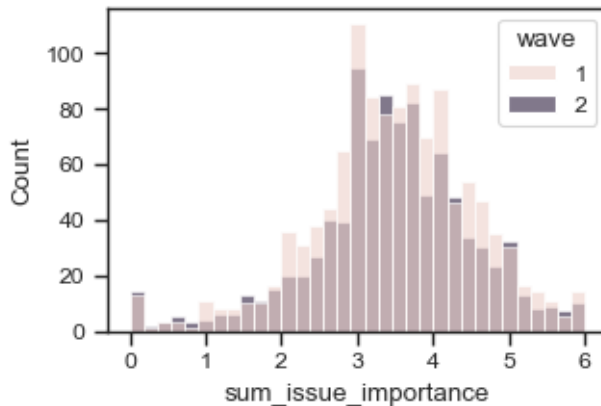
```
[39]: # %%
var = "sum_issue_importance"
fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
df_p[var] = df_p[[f"w_{q}" for q in questions_sc]].sum(axis=1)
sns.histplot(df_p,x=var, hue="wave", ax=axs[0], alpha=0.6)
aa = df_p.pivot_table(index="id", columns="wave", values=var).rename(columns={1:
    ↪ "wave1", 2: "wave2"})
sns.regplot(aa, x="wave1", y="wave2", ax=axs[1], scatter_kws={"s":3, "alpha":0.
    ↪ 5, "color": "grey"})
axs[1].set_aspect("equal")
print(f"correlation wave 2 wave 1 {var}: {aa.corr().values[1,0]}")
fig.tight_layout()
display(df_p[var].describe())
```

correlation wave 2 wave 1 sum_issue_importance: 0.6638441117194276

```
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/1715420902.py:4
: PerformanceWarning: DataFrame is highly fragmented. This is usually the
result of calling `frame.insert` many times, which has poor performance.
Consider joining all columns at once using pd.concat(axis=1) instead. To get a
de-fragmented frame, use `newframe = frame.copy()`
df_p[var] = df_p[[f"w_{q}" for q in questions_sc]].sum(axis=1)
```

count	2131.000000
mean	3.454336
std	1.064184
min	0.000000
25%	2.920000
50%	3.470000
75%	4.120000
max	6.000000

Name: sum_issue_importance, dtype: float64



```
[40]: # %% [markdown]
      # ## Voter Sympathy
```

```
[41]: # %%
from itertools import zip_longest

N = len(partiesVars)
ncols = 2
nrows = (N + 1) // ncols # ceiling division

fig, axs = plt.subplots(
    nrows, ncols,
    figsize=(16/2.54, 12/2.54),
    sharex=True, sharey=True
)
axs_flat = axs.flatten()

for i, p in enumerate(partiesVars):
    ax = axs_flat[i]
    sns.histplot(
        data=df_p,
        x=f"sym_{p}",
        hue="party_close",
        ax=ax,
        alpha=0.6,
        palette=party_cmap,
        multiple="stack",
        bins=np.linspace(-0.01, 1.01, 21),
        legend=(i == N - 1),
        linewidth=0,
```

```

        hue_order=parties_full,
    )
    ax.set_title(f"towards {p} voter", x=0.95, y=0.8,
                color=party_cmap[p], va="top", ha="right")
    ax.set_ylabel("")
    ax.set_xlabel("sympathy towards typical voter" if i >= N - ncols else "")

# Hide unused axes (last cell if N is odd)
for ax in axs_flat[N:]:
    ax.axis("off")

ax = axs_flat[N-1]
sns_legend = ax.get_legend()
handles = sns_legend.legend_handles
labels = [t.get_text() for t in sns_legend.get_texts()]
sns_legend.remove()

# Hide the last axes but keep it as a legend host
empty_ax = axs_flat[N]
empty_ax.axis("off")

empty_ax.legend(
    handles, labels,
    bbox_to_anchor=(0.05, 0.5),
    ncol=2,
    fontsize=6,
    title="party id",
    title_fontsize=6,
    loc="center left",
    borderaxespad=0,
)

display(df_p[[f"sym_{p}" for p in partiesVars]].describe())

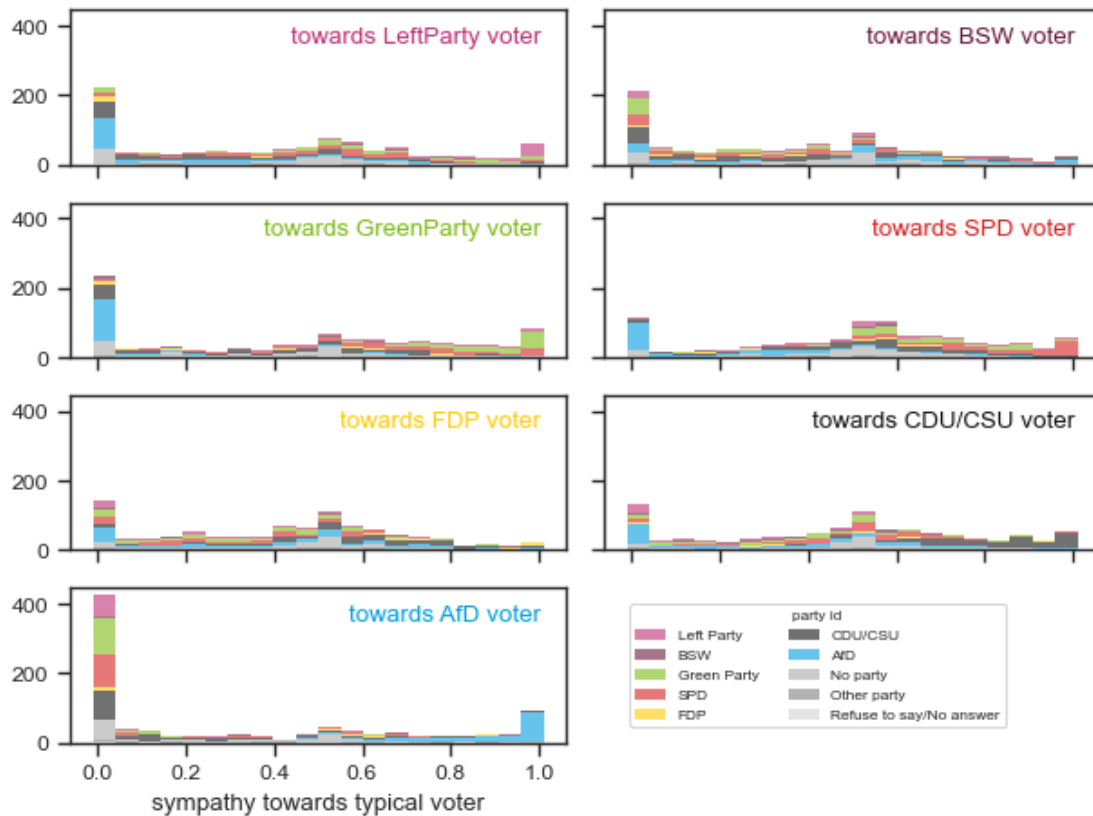
fig.tight_layout()
plt.savefig("sympathy.png", dpi=600)

```

	sym_LeftParty	sym_BSW	sym_GreenParty	sym_SPD	sym_FDP \
count	961.000000	961.000000	961.000000	961.000000	961.000000
mean	0.400708	0.362487	0.449740	0.518803	0.409501
std	0.316579	0.288055	0.345557	0.283031	0.269797
min	0.000000	0.000000	0.000000	0.000000	0.000000
25%	0.070000	0.080000	0.050000	0.350000	0.190000
50%	0.420000	0.360000	0.500000	0.560000	0.440000
75%	0.630000	0.570000	0.740000	0.710000	0.600000
max	1.000000	1.000000	1.000000	1.000000	1.000000

	sym_CDU/CSU	sym_AfD
count	961.000000	961.000000
mean	0.400708	0.362487
std	0.316579	0.288055
min	0.000000	0.000000
25%	0.070000	0.080000
50%	0.420000	0.360000
75%	0.630000	0.570000
max	1.000000	1.000000

count	961.000000	961.000000
mean	0.478127	0.320437
std	0.296390	0.365198
min	0.000000	0.000000
25%	0.250000	0.000000
50%	0.510000	0.120000
75%	0.690000	0.630000
max	1.000000	1.000000



```
[42]: # %% [markdown]
      # # Training
```

```
[54]: var = "attemptsPractice"
      print(f"Fraction of training fails in wave 1: {sum((df_p.
      ↪ wave==1)*(df_p[var]==-999))/sum(df_p.wave==1):.4f}")
      # df_p[["wave", var]].value_counts().reset_index().sort_values("wave")
```

Fraction of training fails in wave 1: 0.2043

```
[55]: # %%
      for var in ["attemptsPractice"]:
```



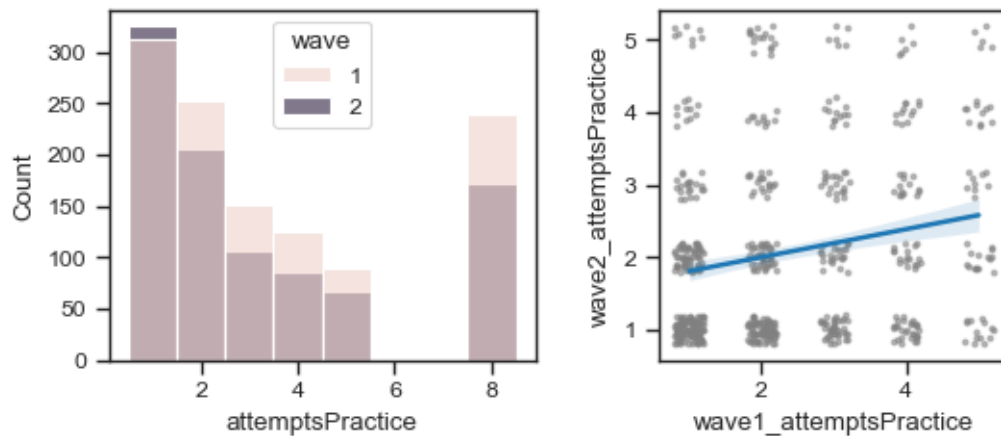
```

fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))

sns.histplot(df_p.replace({-999:8}), x=var, hue="wave", ax=axs[0], bins=0.
↪5+np.arange(0,9), alpha=0.6)
sns.regplot(df_p_bothwaves[["wave1_"+var, "wave2_"+var]].replace({-999:np.
↪nan}), x="wave1_"+var, y="wave2_"+var, ax=axs[1], scatter_kws={"s":3,
↪"alpha":0.5, "color":"grey"}, y_jitter=0.2*(var=="attemptsPractice"),
↪x_jitter=0.2*(var=="attemptsPractice"))
axs[1].set_aspect("equal")
print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
↪'wave1_'+var]].replace({-999:np.nan}).corr().values[1,0]}")

```

correlation wave 2 wave 1 attemptsPractice: 0.20324312818343532

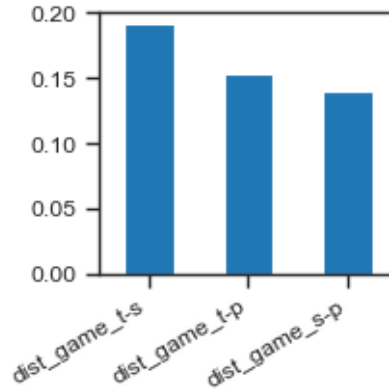


```

[56]: # %%
display(df_p[[f"dist_game_{a[0]}-{b[0]}" for a,b in
↪combinations(practice_game_dots, 2)]].describe())
fig = plt.figure(figsize=(2,2))
df_p[[f"dist_game_{a[0]}-{b[0]}" for a,b in combinations(practice_game_dots,
↪2)]].mean().plot.bar()
fig.autofmt_xdate()

```

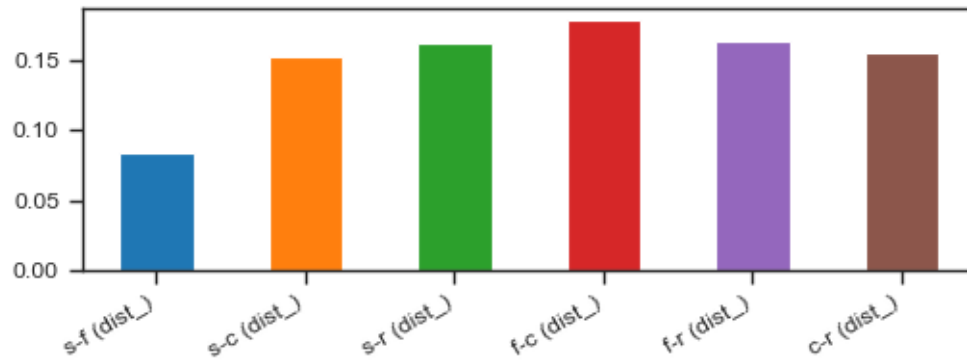
	dist_game_t-s	dist_game_t-p	dist_game_s-p
count	2131.000000	2131.000000	2131.000000
mean	0.191509	0.152629	0.139701
std	0.091240	0.085501	0.080237
min	0.001818	0.001286	0.001286
25%	0.124870	0.089510	0.077762
50%	0.180742	0.135390	0.124734
75%	0.252008	0.198827	0.184760
max	0.501818	0.501818	0.501435



```
[57]: # %%
vars = [f"dist_practice_{a[0]}-{b[0]}" for a,b in combinations(practice_training_dots, 2)]
display(df_p[vars].describe())
fig = plt.figure(figsize=(6,2))
df_p[vars].mean().plot.bar(color=plt.get_cmap("tab10").colors)
plt.xticks(plt.gca().get_xticks(), labels=[s[-3:]+ " (" +s[:5]+")" for s in vars])
fig.autofmt_xdate()
```

	dist_practice_s-f	dist_practice_s-c	dist_practice_s-r \
count	2131.000000	2131.000000	2131.000000
mean	0.084429	0.151904	0.161647
std	0.054933	0.075922	0.082202
min	0.002875	0.014374	0.011499
25%	0.047304	0.093123	0.095583
50%	0.068188	0.137276	0.143576
75%	0.104395	0.194868	0.217880
max	0.425764	0.487494	0.501818

	dist_practice_f-c	dist_practice_f-r	dist_practice_c-r
count	2131.000000	2131.000000	2131.000000
mean	0.178163	0.164070	0.155472
std	0.078194	0.071335	0.074121
min	0.016109	0.010602	0.018408
25%	0.117632	0.110763	0.096466
50%	0.167312	0.154674	0.145028
75%	0.230376	0.207524	0.200532
max	0.501818	0.488367	0.489965



```
[58]: # %%
df_p[["wave", "passed_practice_sanity"]].replace({np.nan:"failed training"}).
↳groupby("wave")["passed_practice_sanity"].value_counts().reset_index()
```

```
[58]: wave passed_practice_sanity count
0      1                1.0      925
1      1      failed training    239
2      1                0.0        6
3      2                1.0     776
4      2      failed training    172
5      2                0.0        13
```

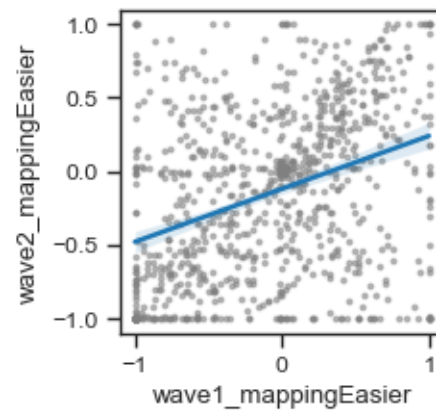
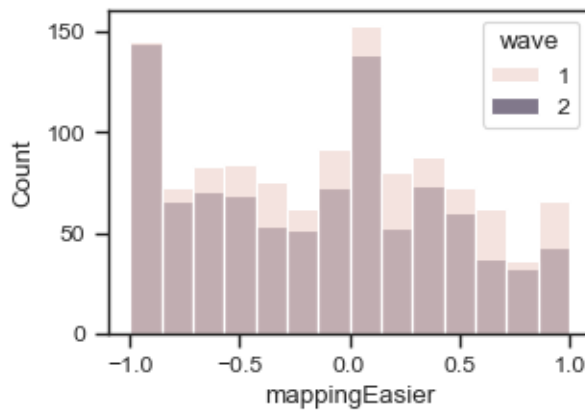
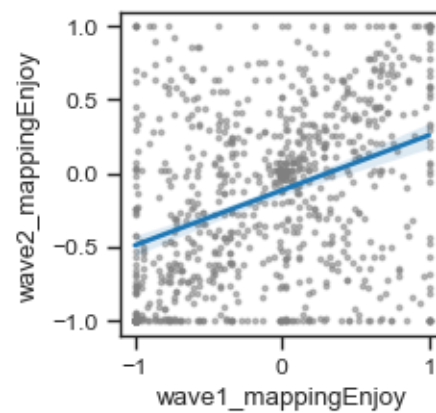
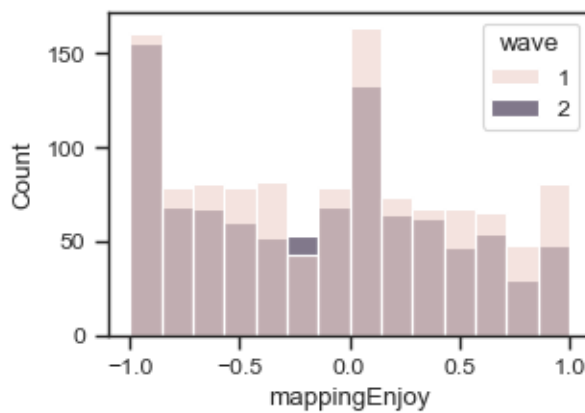
```
[59]: # %% [markdown]
# ## Mapping
```

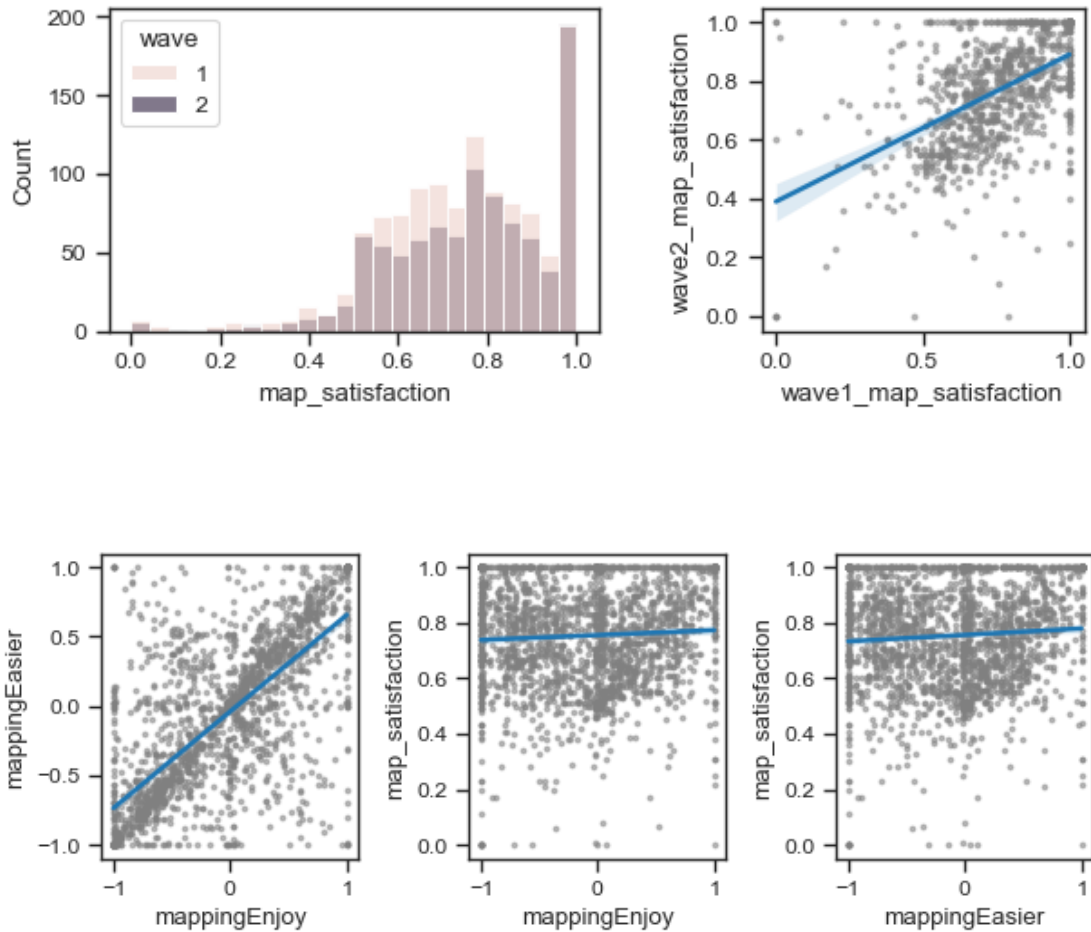
```
[60]: # %%
vars = ["mappingEnjoy", "mappingEasier", "map_satisfaction"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p,x=var, hue="wave", ax=axs[0], alpha=0.6)
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],
↳scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
↳'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

display(df_p[vars].describe())
fig, axs = plt.subplots(1,3, figsize=(16/2.54,6/2.54))
for (var1, var2), ax in zip(combinations(vars,2), axs.flatten()):
    sns.regplot(df_p, x=var1, y=var2, ax=ax, scatter_kws={"s":3, "alpha":0.5,
↳"color":"grey"})
fig.tight_layout()
```

correlation wave 2 wave 1 mappingEnjoy: 0.3746722332641922
correlation wave 2 wave 1 mappingEasier: 0.3559971135908032
correlation wave 2 wave 1 map_satisfaction: 0.5124366548145483

	mappingEnjoy	mappingEasier	map_satisfaction
count	2131.000000	2131.000000	2131.000000
mean	-0.119962	-0.121192	0.753867
std	0.593370	0.573330	0.185781
min	-1.000000	-1.000000	0.000000
25%	-0.640000	-0.620000	0.630000
50%	-0.040000	-0.040000	0.770000
75%	0.320000	0.320000	0.900000
max	1.000000	1.000000	1.000000





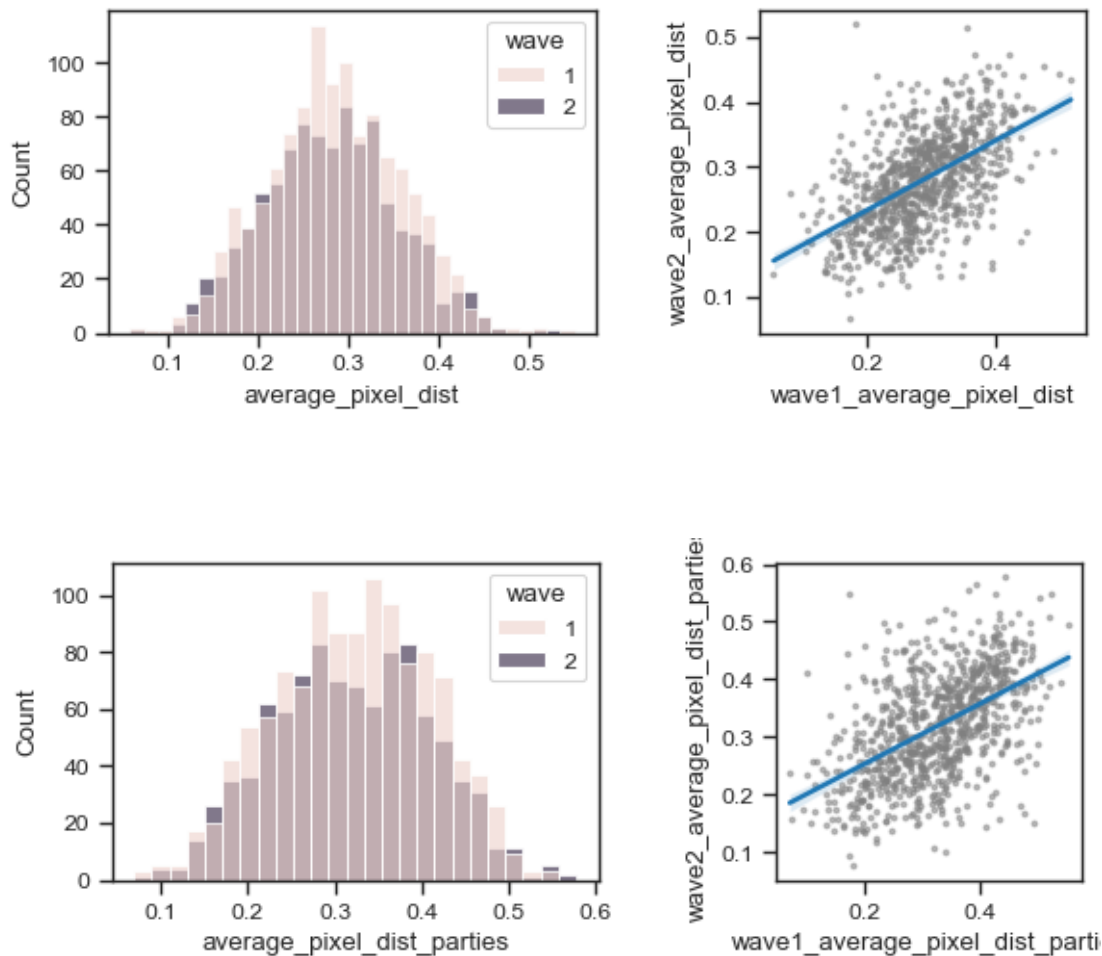
```
[61]: # %%
vars = ["average_pixel_dist", "average_pixel_dist_parties"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p, x=var, hue="wave", ax=axs[0], alpha=0.6)
    sns.regplot(df_p_bothwaves, x="wave1_"+var, y="wave2_"+var, ax=axs[1],
    ↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

display(df_p[vars].describe())
```

```
correlation wave 2 wave 1 average_pixel_dist: 0.544919637973932
correlation wave 2 wave 1 average_pixel_dist_parties: 0.5180154828198414
```

```
average_pixel_dist  average_pixel_dist_parties
```

count	2131.000000	2131.000000
mean	0.281627	0.320045
std	0.073028	0.089704
min	0.057501	0.068942
25%	0.230760	0.254271
50%	0.280162	0.320929
75%	0.331161	0.387069
max	0.549351	0.577399



```
[62]: # %%
vars = ["average_pixel_dist", "average_pixel_dist_parties"]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p.loc[df_p["wave"]==2], x=var, hue="treatment_wave2",
    ↪ax=axs[0], alpha=0.6)
    for t in [0,1]:
```

```

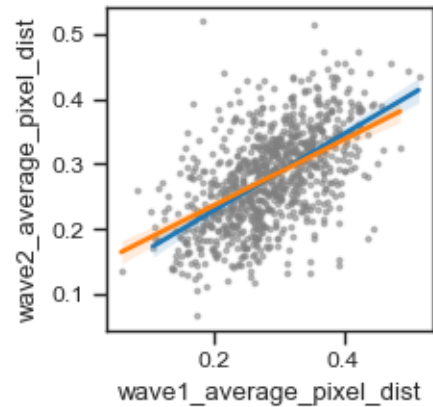
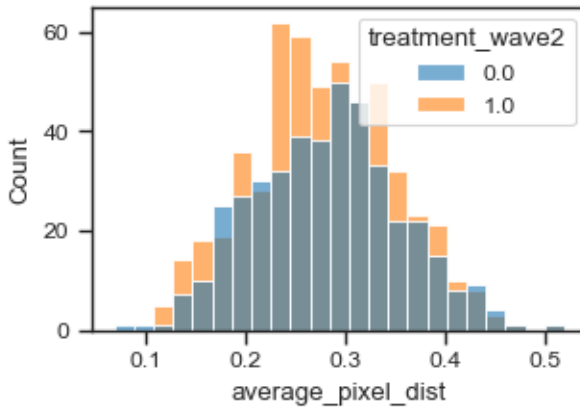
sns.regplot(df_p_bothwaves.loc[df_p_bothwaves.
↪wave2_treatment_wave2==t], x="wave1_"+var, y="wave2_"+var, ax=axes[1],
↪scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
axes[1].set_aspect("equal")
print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
↪'wave1_'+var]].corr().values[1,0]}")
fig.tight_layout()

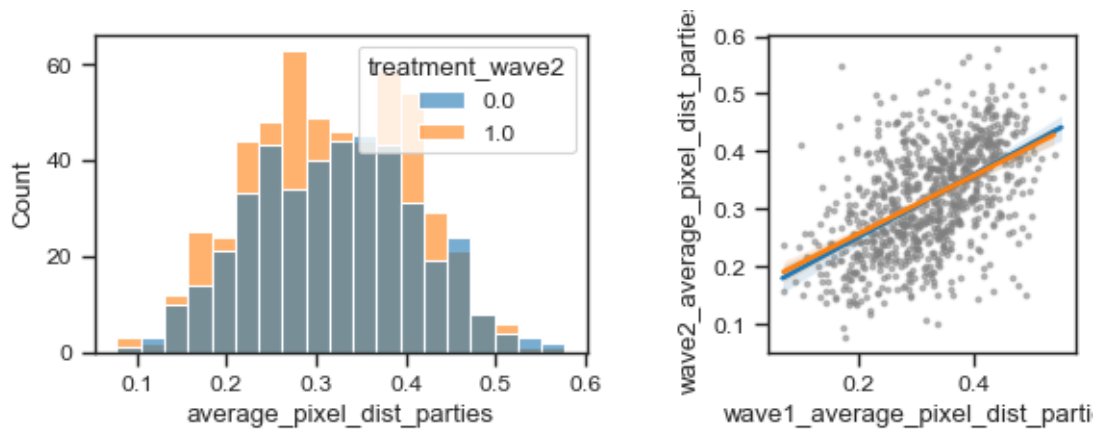
display(df_p[vars].describe())

```

correlation wave 2 wave 1 average_pixel_dist: 0.544919637973932
correlation wave 2 wave 1 average_pixel_dist_parties: 0.5180154828198414

	average_pixel_dist	average_pixel_dist_parties
count	2131.000000	2131.000000
mean	0.281627	0.320045
std	0.073028	0.089704
min	0.057501	0.068942
25%	0.230760	0.254271
50%	0.280162	0.320929
75%	0.331161	0.387069
max	0.549351	0.577399





```
[63]: # %% [markdown]
# # Opinions
```

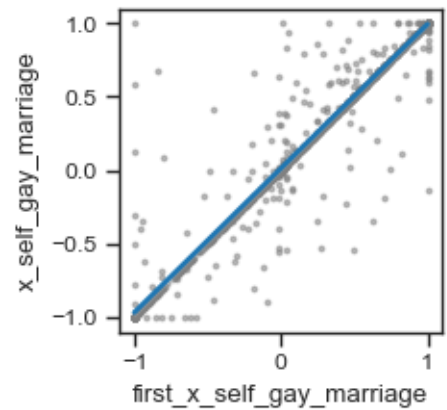
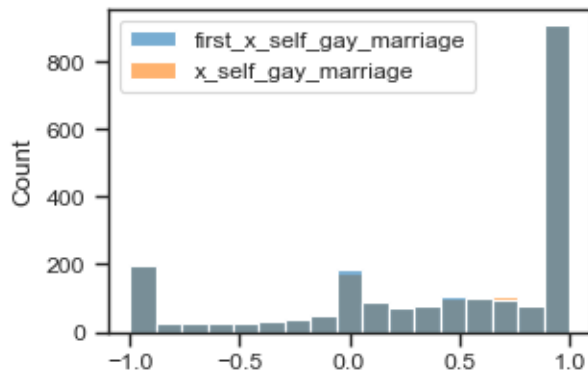
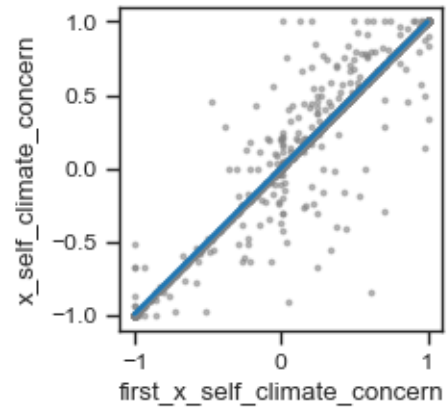
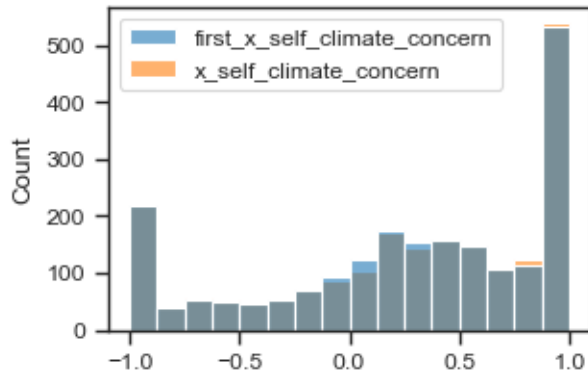
```
[64]: # %%
vars = [f"x_self_{q}" for q in questions_sc]
varsPrior = [f"first_x_self_{q}" for q in questions_sc]
wave = 2
for var, varp in zip(vars, varsPrior):
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p[[varp, var]], ax=axs[0], alpha=0.6)
    sns.regplot(df_p, x=varp, y=var, ax=axs[1], scatter_kws={"s":3, "alpha":0.
↪5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave {wave} {var} and {varp}: {df_p[[varp, var]].corr().
↪values[1,0]}")
    fig.tight_layout()

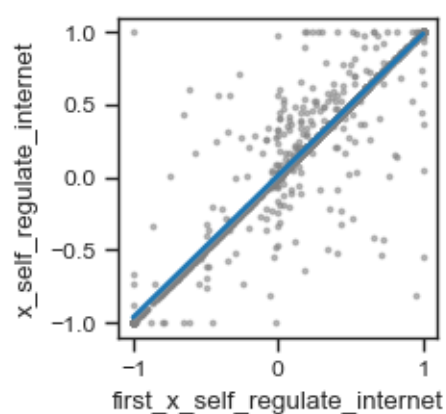
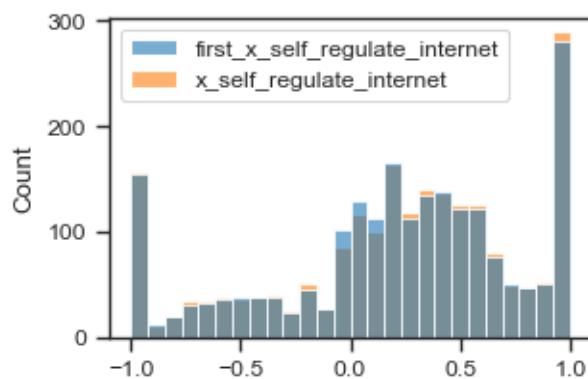
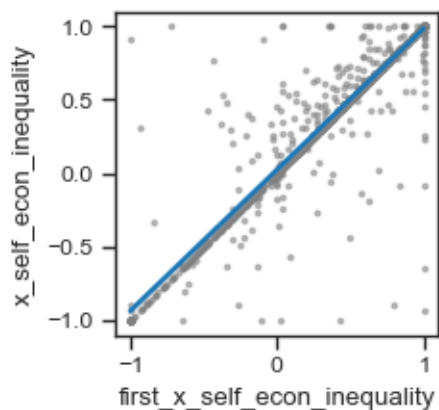
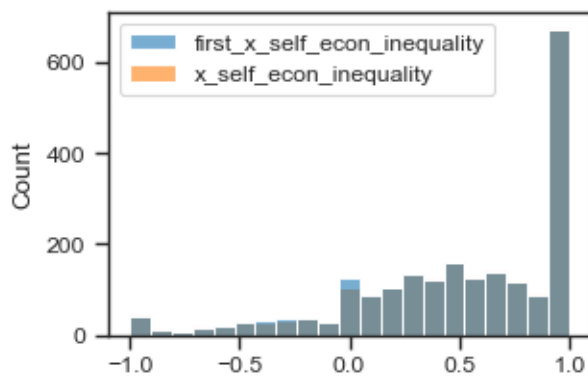
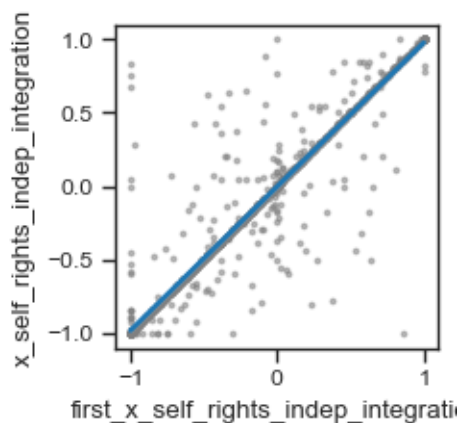
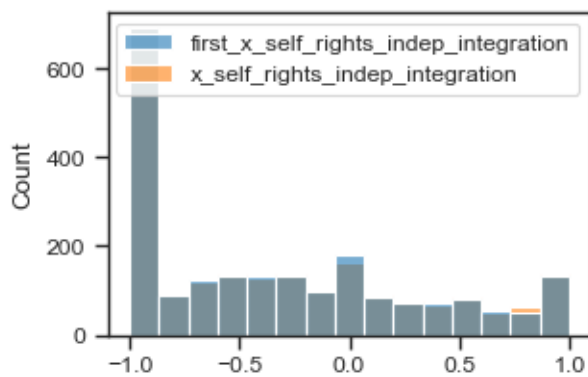
print(f"number of people who changed their opinions: {dict(zip(questions_sc,
↪((np.abs(df_p[[var for var in vars]].values - df_p[[varp for varp in
↪varsPrior]].values)>0).sum(axis=0))))}")
```

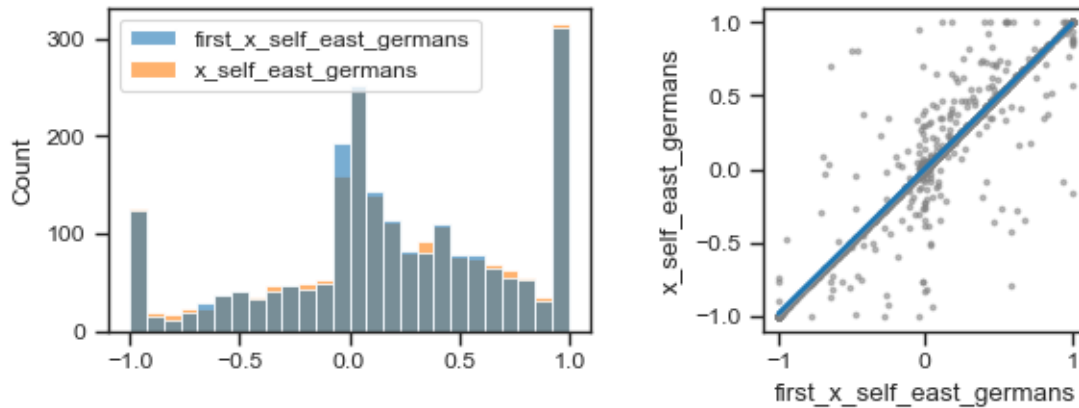
```
correlation wave 2 x_self_climate_concern and first_x_self_climate_concern:
0.9859844235430527
correlation wave 2 x_self_gay_marriage and first_x_self_gay_marriage:
0.9813192325633138
correlation wave 2 x_self_rights_indep_integration and
first_x_self_rights_indep_integration: 0.9728781866327584
correlation wave 2 x_self_econ_inequality and first_x_self_econ_inequality:
0.9550145082719753
correlation wave 2 x_self_regulate_internet and first_x_self_regulate_internet:
0.9649260664845698
correlation wave 2 x_self_east_germans and first_x_self_east_germans:
```


0.9702255933872661

```
number of people who changed their opinions: {'climate_concern': np.int64(222),  
'gay_marriage': np.int64(195), 'rights_indep_integration': np.int64(214),  
'econ_inequality': np.int64(229), 'regulate_internet': np.int64(260),  
'east_germans': np.int64(270)}
```



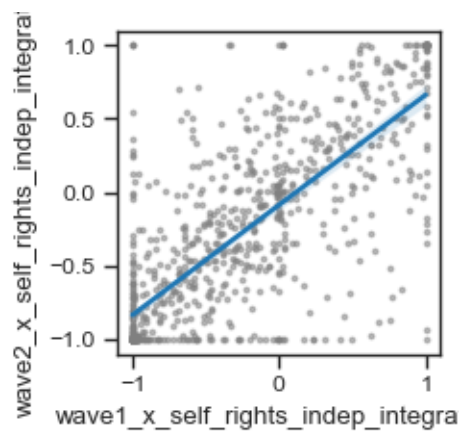
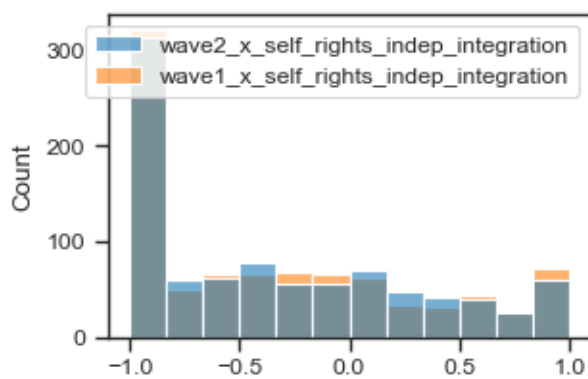
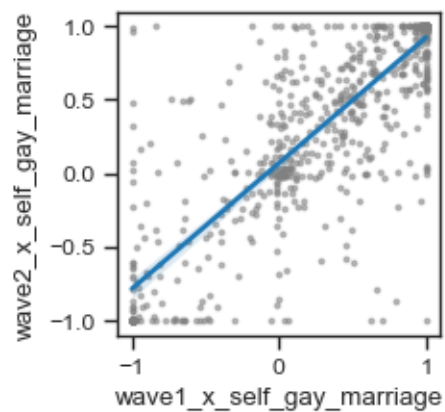
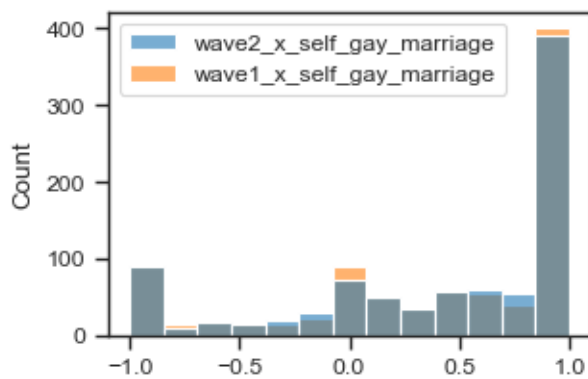
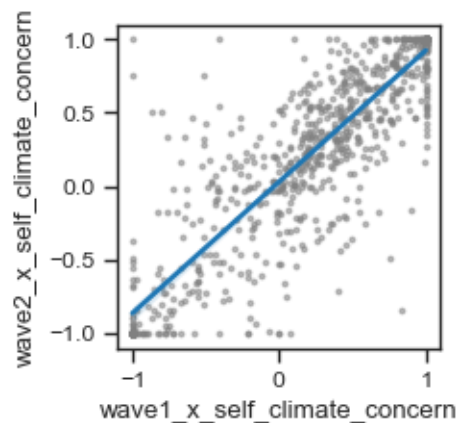
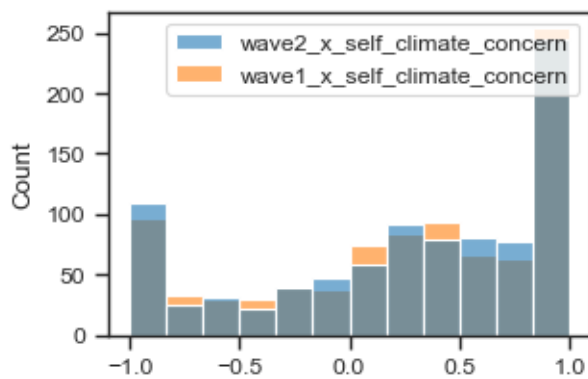


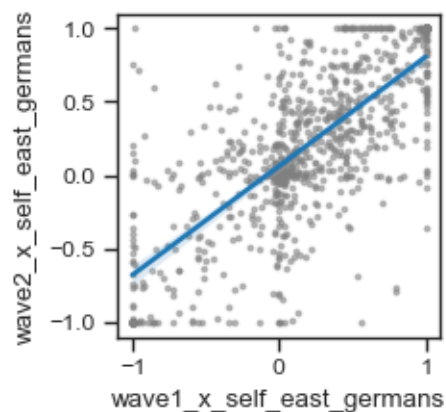
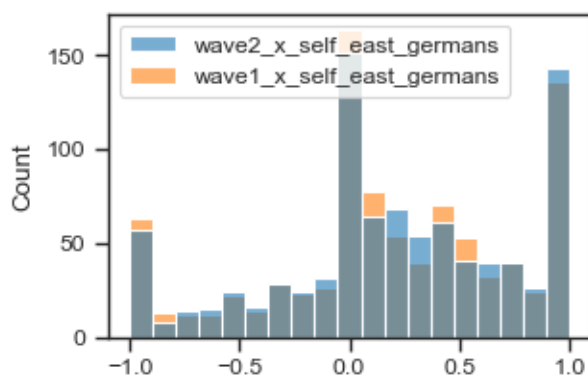
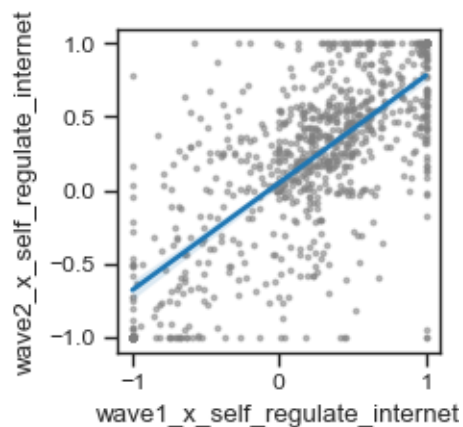
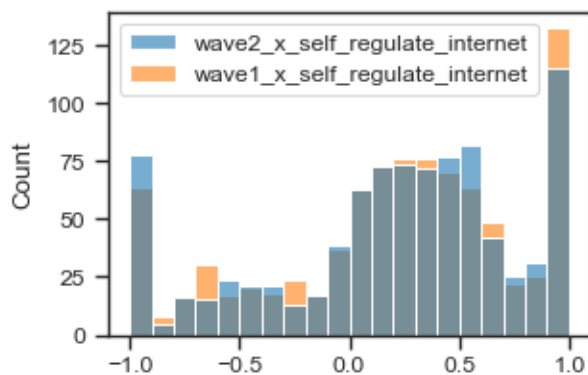
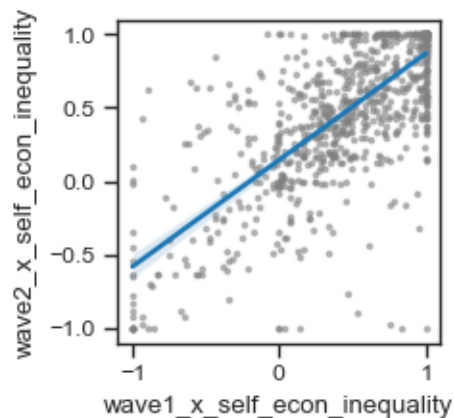
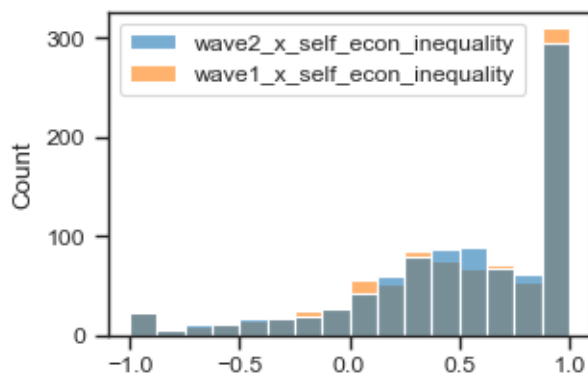


```
[65]: # %%
vars = [f"x_self_{q}" for q in questions_sc]
for var in vars:
    fig, axs = plt.subplots(1,2, figsize=(16/2.54,6/2.54))
    sns.histplot(df_p_bothwaves[["wave2_"+var, "wave1_"+var]], ax=axs[0],
    ↪alpha=0.6)
    sns.regplot(df_p_bothwaves[["wave2_"+var, "wave1_"+var]], x="wave1_"+var,
    ↪y="wave2_"+var, ax=axs[1], scatter_kws={"s":3, "alpha":0.5, "color":"grey"})
    axs[1].set_aspect("equal")
    print(f"correlation wave 2 wave 1 {var}: {df_p_bothwaves[['wave2_'+var,
    ↪'wave1_'+var]].corr().values[1,0]}")
    fig.tight_layout()

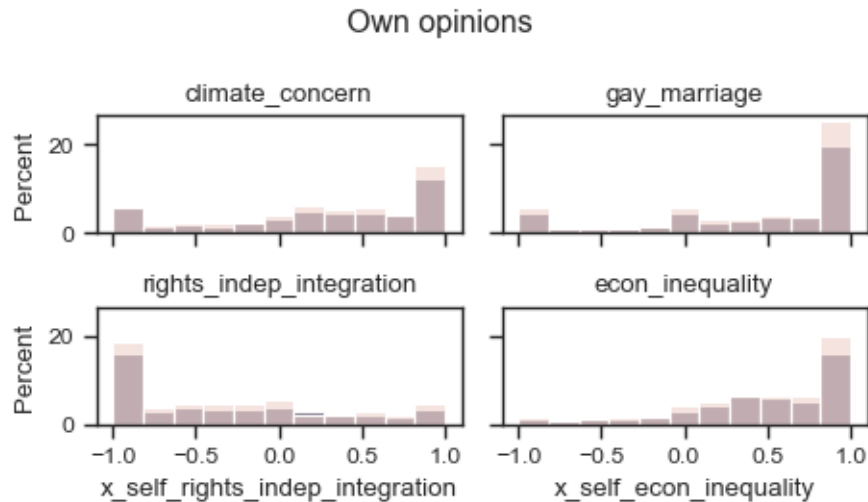
# display(df_p[[var for var in vars]].describe())
```

```
correlation wave 2 wave 1 x_self_climate_concern: 0.8879664029577369
correlation wave 2 wave 1 x_self_gay_marriage: 0.8589363352551868
correlation wave 2 wave 1 x_self_rights_indep_integration: 0.7655027089300674
correlation wave 2 wave 1 x_self_econ_inequality: 0.7500067135841224
correlation wave 2 wave 1 x_self_regulate_internet: 0.7286592697676602
correlation wave 2 wave 1 x_self_east_germans: 0.7475457548236663
```



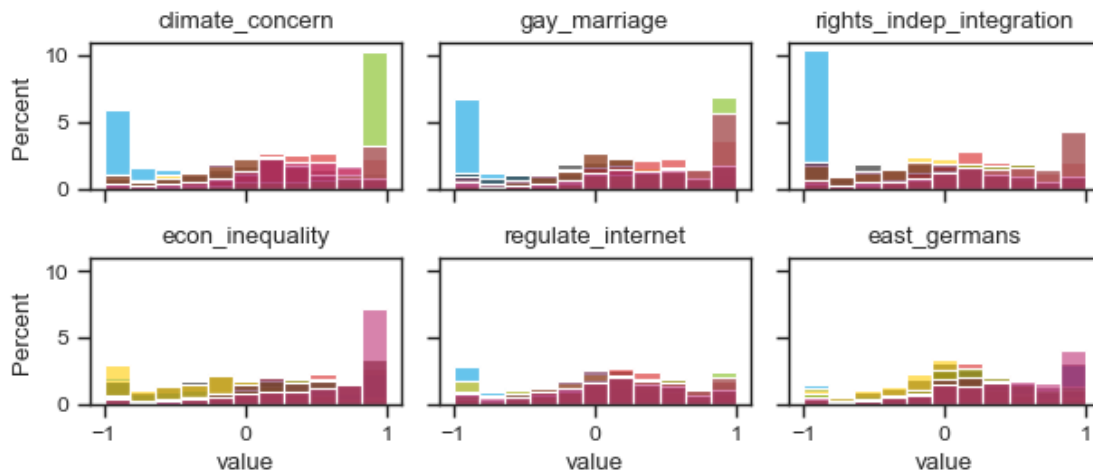


```
[66]: # %%
fig, axs = plt.subplots(2,2, figsize=(12/2.54,7/2.54), sharex=True, sharey=True)
for ax, q in zip(axs.flatten(), questions_sc):
    sns.histplot(df_p, x = f"x_self_{q}", hue="wave", bins=11, binrange=(-1,1),
    ↪ax=ax, alpha=0.6, legend=False, stat="percent", multiple="layer", kde=False)
    ax.set_title(q)
fig.suptitle("Own opinions")
fig.tight_layout()
```



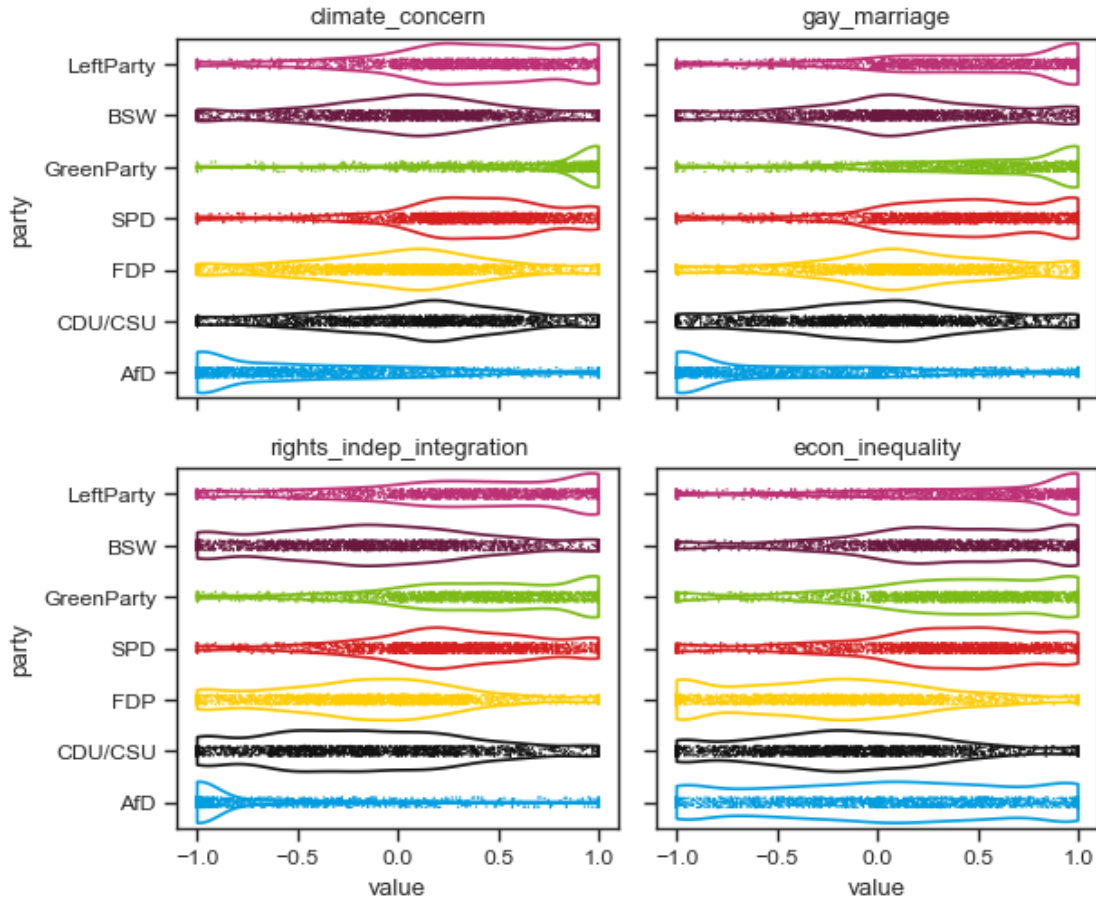
```
[67]: # %%
fig, axs = plt.subplots(2,3, figsize=(16/2.54,8/2.54), sharex=True, sharey=True)
for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"x_{p}_{q}" for p in partiesVars]].melt(var_name="party", )
    a["party"] = a["party"].apply(lambda x: x.split("_")[1])
    sns.histplot(a, x="value", hue="party", palette=party_cmap,
    ↪hue_order=partiesVars, bins=11, binrange=(-1,1), ax=ax, alpha=0.6,
    ↪legend=False, stat="percent", multiple="layer", kde=False)
    ax.set_title(q)
fig.suptitle("What are the opinions of a typical voter of party...?")
fig.tight_layout()
```

What are the opinions of a typical voter of party...?



```
[68]: # %%
fig, axs = plt.subplots(2,2, figsize=(16/2.54,14/2.54), sharex=True,
    ↳sharey=True)
fig.suptitle("What are the opinions of a typical voter of party...?")
for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"x_{p}_{q}" for p in partiesVars]].melt(var_name="party")
    a["party"] = a["party"].apply(lambda x: x.split("_")[1])
    sns.violinplot(a, x="value", y="party", hue="party", fill=False,
    ↳inner=None, palette=party_cmap, hue_order=partiesVars, ax=ax, legend=False,
    ↳cut=0, )
    sns.stripplot(a, x="value", y="party", hue="party", palette=party_cmap,
    ↳hue_order=partiesVars, ax=ax, legend=False, size=1)
    ax.set_title(q)
fig.tight_layout()
```

What are the opinions of a typical voter of party...?



```
[69]: # %% [markdown]
# ## Std Dev of Opinions in social circles
```

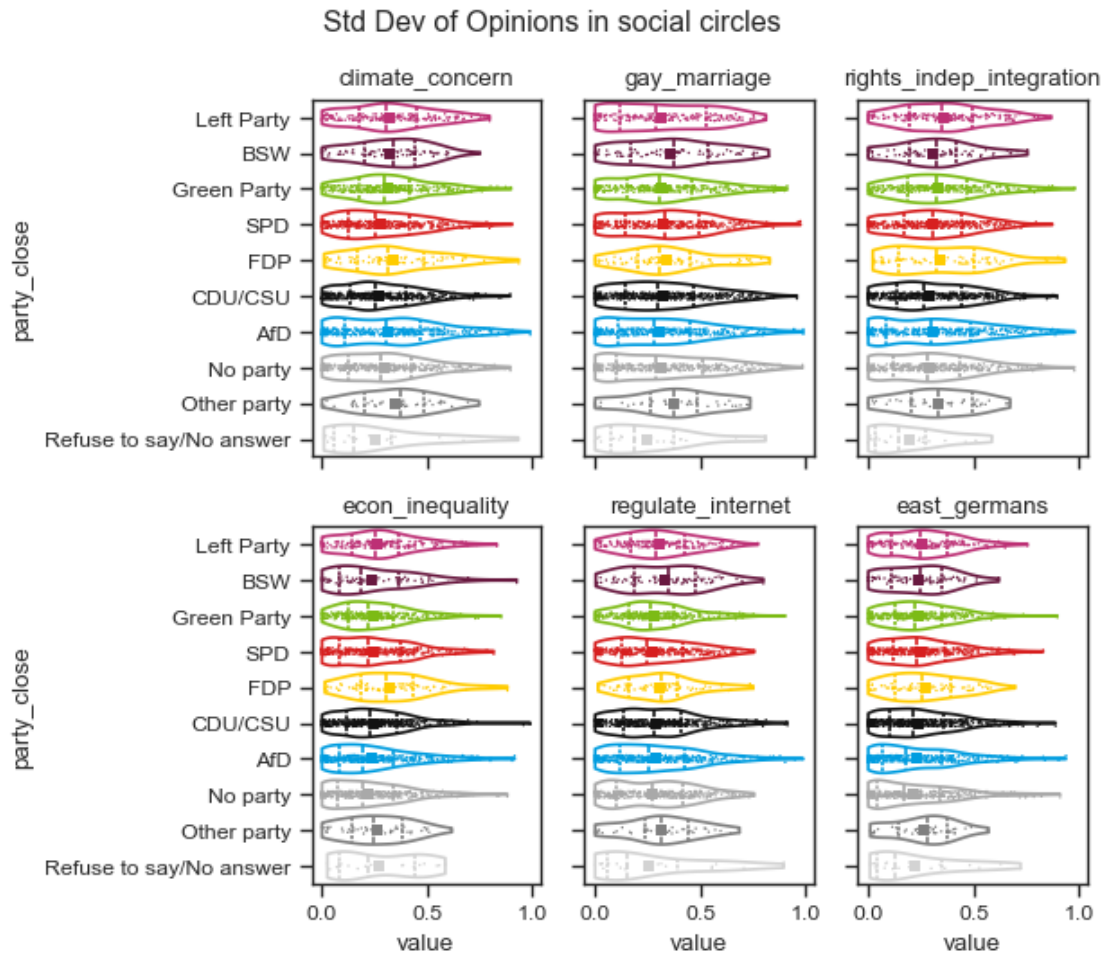
```
[70]: # %%
fig, axs = plt.subplots(2,3, figsize=(16/2.54,14/2.54), sharex=True,
    ↪sharey=True)
fig.suptitle("Std Dev of Opinions in social circles")
for ax, q in zip(axs.flatten(), questions_sc):
    a = df_p[[f"std_socialCircle_ops_{q}"]+[f"party_close"]].
    ↪melt(var_name="social_circle", id_vars=[f"party_close"])
    sns.violinplot(a, x="value", y=f"party_close", hue=f"party_close",
    ↪fill=False, palette=party_cmap, hue_order=parties_full, ax=ax,
    ↪legend=False, cut=0, inner="quart", order=parties_full)
    sns.stripplot(a, x="value", y=f"party_close", hue=f"party_close",
    ↪palette=party_cmap, hue_order=parties_full, ax=ax, legend=False, size=1)
```



```

sns.stripplot(a.groupby(f"party_close")["value"].mean().reset_index(),
x="value", y=f"party_close", hue=f"party_close", palette=party_cmap,
hue_order=parties_full, ax=ax, legend=False, size=5, marker="s")
ax.set_title(q)
fig.tight_layout()

```



```

[71]: # %% [markdown]
      # ### Treatment

```

```

[72]: # %%
      print(f"Treatment: {df_p['treatment_wave2'].value_counts().to_dict()}")

```

Treatment: {1.0: 539, 0.0: 422}

```

[73]: # %%
      fig, ax = plt.subplots(1,1, figsize=(16/2.54,14/2.54))
      ax.set_title("Std Dev of Opinions in social circles")

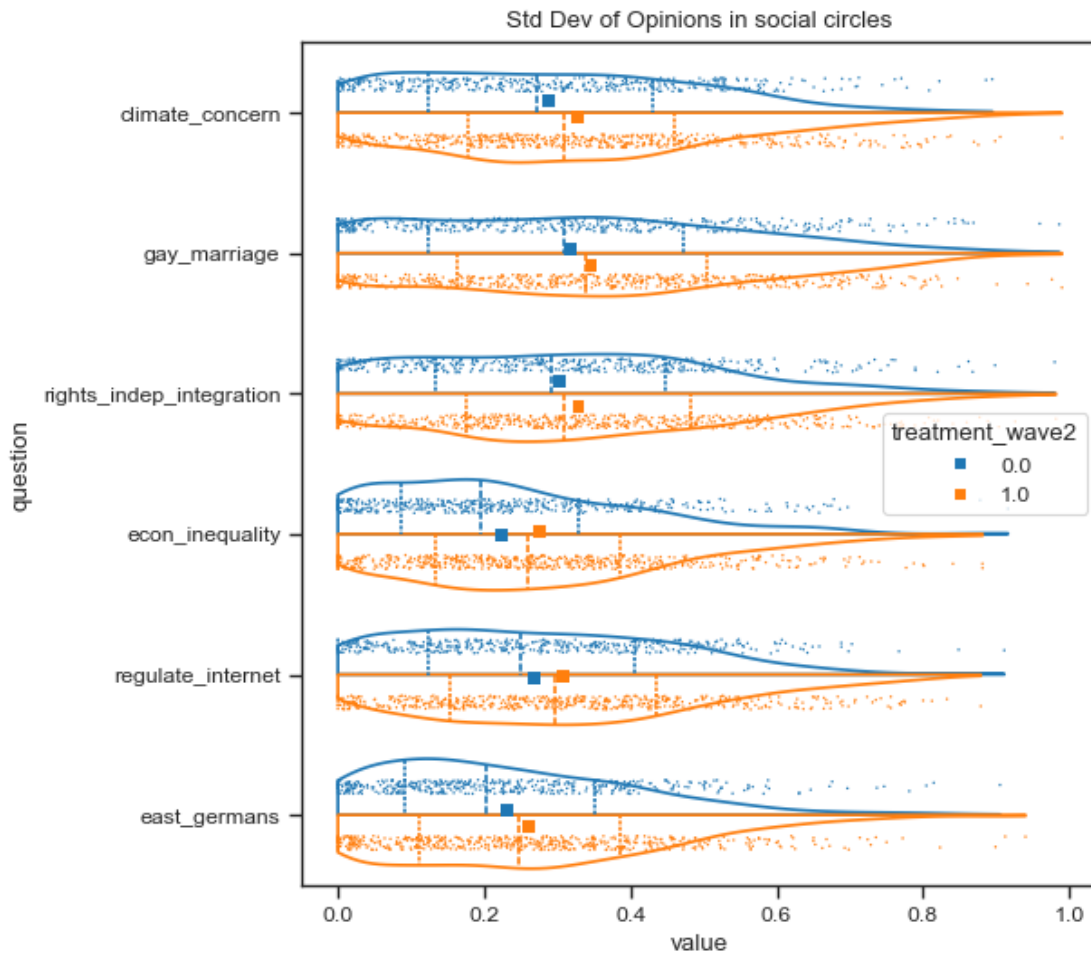
```

```

a = df_p[[f"std_socialCircle_ops_{q}" for q in
↳questions_sc]+[f"treatment_wave2"]].melt(var_name="question",
↳id_vars=[f"treatment_wave2"])
a["question"] = a["question"].apply(lambda x: "_".join(x.split("_")[3:]))
a = a.dropna()
print(len(a))
sns.violinplot(a, x="value", y="question", split=True, hue="treatment_wave2",
↳fill=False, ax=ax, legend=False, cut=0, inner="quart")
sns.stripplot(a, x="value", y="question", hue=f"treatment_wave2", ax=ax,
↳legend=False, size=1, dodge=True)
sns.stripplot(a.groupby([f"treatment_wave2", "question"])["value"].mean().
↳reset_index(), x="value", y=f"question", hue=f"treatment_wave2", ax=ax,
↳legend=True, size=5, marker="s")
fig.tight_layout()

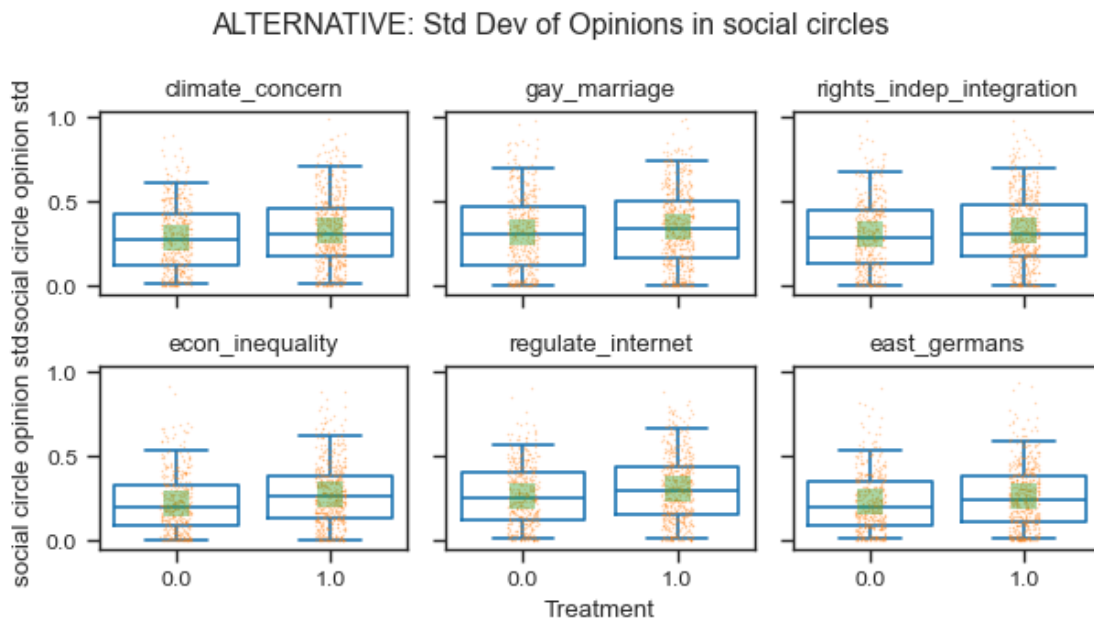
```

5766



```
[74]: # %%
fig, axs = plt.subplots(2,3, sharex=True, sharey=True)
w=2
fig.suptitle("ALTERNATIVE: Std Dev of Opinions in social circles")

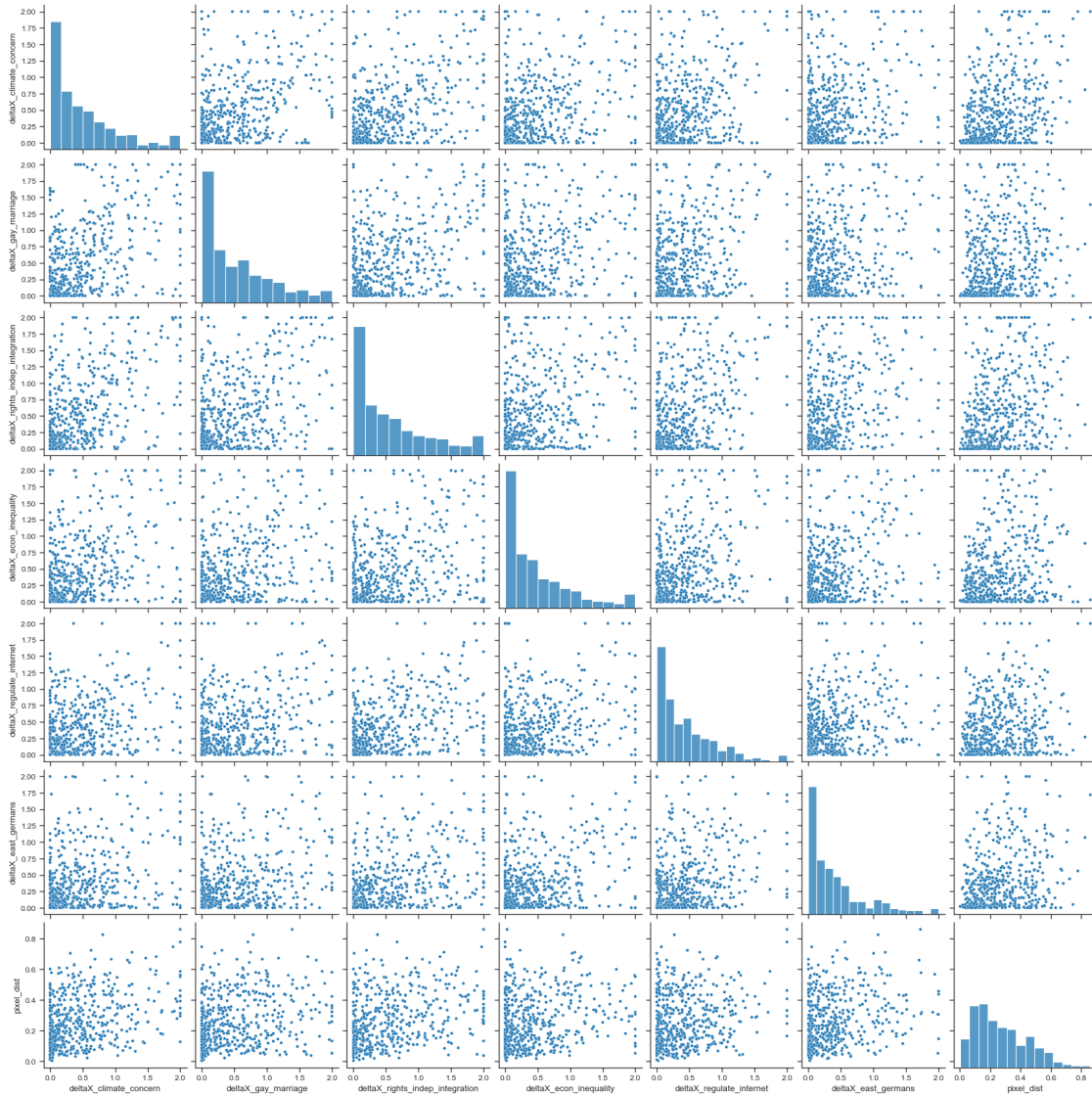
for ax, qq in zip(axs.flatten(), questions_sc):
    sns.boxplot(df_p, x="treatment_wave2", y=f"std_socialCircle_ops_{qq}",
    ↪ax=ax, fill=False, whis=[5,95], fliersize=0)
    sns.stripplot(df_p, x="treatment_wave2", y=f"std_socialCircle_ops_{qq}",
    ↪ax=ax, size=1, alpha=0.3)
    sns.stripplot(df_p.groupby("treatment_wave2")[f"std_socialCircle_ops_{qq}"].
    ↪mean().reset_index(), x="treatment_wave2", y=f"std_socialCircle_ops_{qq}",
    ↪ax=ax, size=10, marker="s", alpha=0.4)
    ax.set_title(qq)
    ax.set_xlabel("")
    ax.set_ylabel("social circle opinion std")
axs[-1,1].set_xlabel("Treatment")
fig.tight_layout()
```



```
[75]: # %% [markdown]
# # DISTANCES
```

```
[76]: # %%
diff_cols = [f'deltaX_{q}' for q in questions_sc] + ['pixel_dist']
sns.pairplot(df_diff.sample(500)[diff_cols], plot_kws={'size':0.1}, )
```

[76]: <seaborn.axisgrid.PairGrid at 0x35bd7dd30>



```
[86]: # %%
fig, axs = plt.subplots(2,2, figsize=(7/2.54, 7/2.54))
df_diff["dotype"] = df_diff.apply(lambda x: "personal" if (("reference" in
    ↪x['dot1'] or "self" in x["dot1"]) and ("reference" in x['dot2'] or "self" in
    ↪x["dot2"])) else "voter", axis=1)

hue= None#"dotype"
hue_order = None#["voter", "personal"] #[2,1]
mult = "stack"
sns.histplot(df_diff, x="pairwise_similarity", hue=hue, palette="Set1",
    ↪hue_order=hue_order, ax=axs[0,0], multiple=mult, kde=True, bins=21)
```

```

axs[0,0].set_xlabel("pairwise similarity")

sns.histplot(df_diff, x="pixel_dist", hue=hue, palette="Set1",
             hue_order=hue_order, ax=axs[0,1], multiple=mult, kde=True, bins=21)
axs[0,1].set_xlabel("map distance")

sns.histplot(df_diff, x="sympathy", hue=hue, palette="Set1",
             hue_order=hue_order, ax=axs[1,0], multiple=mult, kde=True, bins=21)
axs[1,0].set_xlabel("likability [voters]")

sns.histplot(df_diff, x="socialCloseness", hue=hue, palette="Set1",
             hue_order=hue_order, ax=axs[1,1], multiple=mult, kde=True, bins=21)
axs[1,1].set_xlabel("closeness [contacts]")

for ax in axs.flatten():
    ax.set_ylabel("")
    ax.set_yticks([])

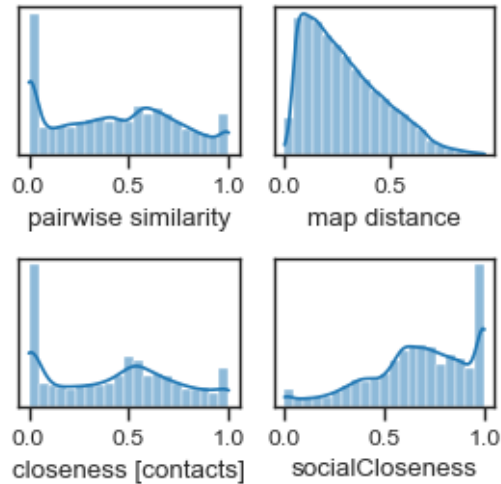
fig.tight_layout()

```

```

/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/2748574209.py:8
: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.
  sns.histplot(df_diff, x="pairwise_similarity", hue=hue, palette="Set1",
hue_order=hue_order, ax=axs[0,0], multiple=mult, kde=True, bins=21)
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/2748574209.py:1
1: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.
  sns.histplot(df_diff, x="pixel_dist", hue=hue, palette="Set1",
hue_order=hue_order, ax=axs[0,1], multiple=mult, kde=True, bins=21)
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/2748574209.py:1
4: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.
  sns.histplot(df_diff, x="sympathy", hue=hue, palette="Set1",
hue_order=hue_order, ax=axs[1,0], multiple=mult, kde=True, bins=21)
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/2748574209.py:1
7: UserWarning: Ignoring `palette` because no `hue` variable has been assigned.
  sns.histplot(df_diff, x="socialCloseness", hue=hue, palette="Set1",
hue_order=hue_order, ax=axs[1,1], multiple=mult, kde=True, bins=21)

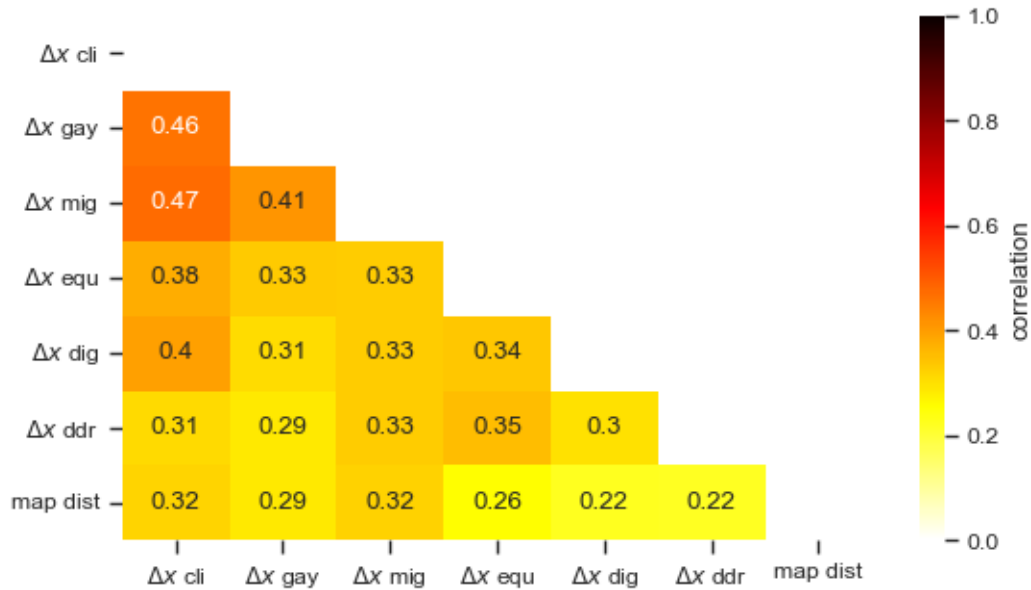
```



```
[90]: # %%
plt.figure()
diff_cols = [f'deltaX_{q}' for q in questions_sc] + ['pixel_dist']
diff_cols_names = [f'$\Delta x$ {q}' for q in qs] + ['map dist']
mask = np.triu(np.ones_like(df_diff[diff_cols].corr()))
colsNames = dict(zip(diff_cols, diff_cols_names))
sns.heatmap(df_diff[diff_cols].corr().rename(columns=colsNames,
↪index=colsNames), annot=True, cmap="hot_r", vmax=1, vmin=0, mask=mask,
↪cbar_kws={'label':"correlation"})

<>:4: SyntaxWarning: invalid escape sequence '\D'
<>:4: SyntaxWarning: invalid escape sequence '\D'
/var/folders/07/c88jy3fd1lbdwnzsr8qy7w0w0000gp/T/ipykernel_82816/2852265211.py:4
: SyntaxWarning: invalid escape sequence '\D'
    diff_cols_names = [f'$\Delta x$ {q}' for q in qs] + ['map dist']
```

[90]: <Axes: >

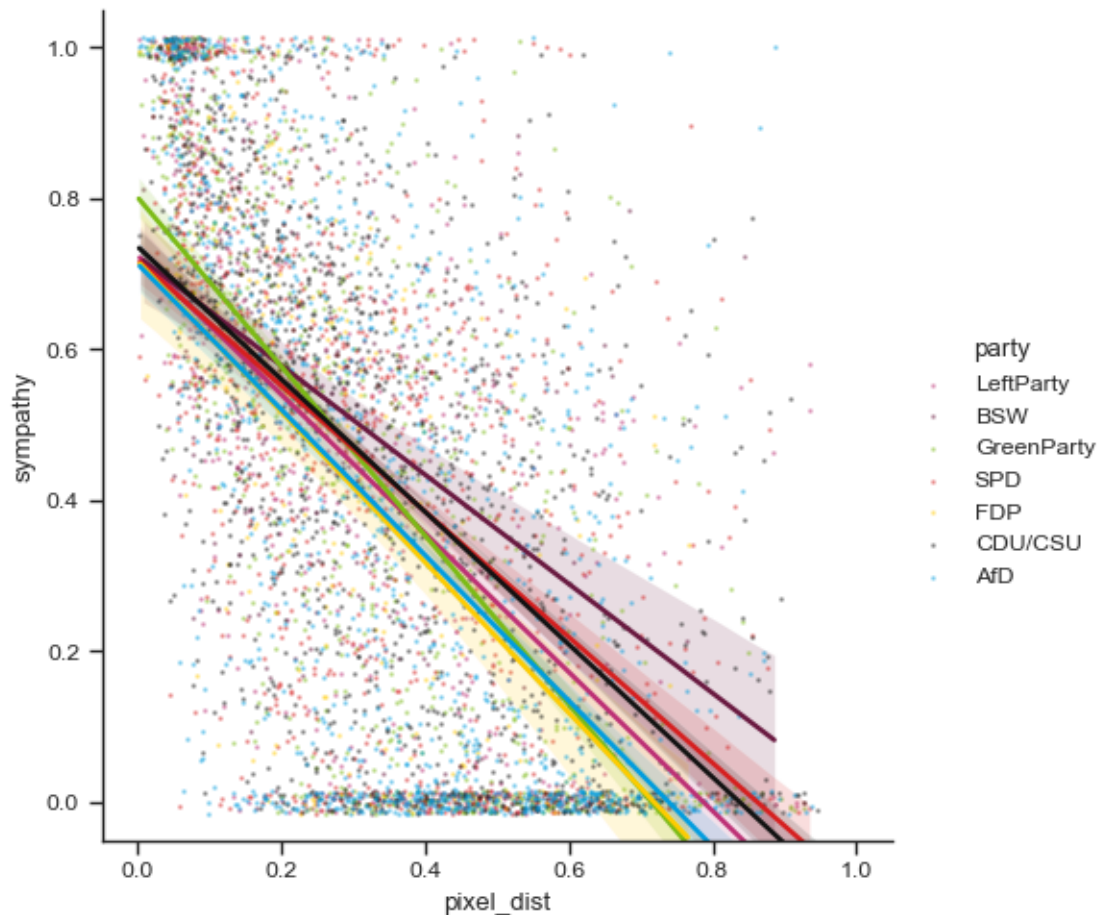


```
[91]: # %%
plt.figure()
print(f"Correlation between Sympathy and pixel distance of a voter:␣
↪{df_diff[['sympathy']+['pixel_dist']].corr().iloc[0,1]}")
sns.lmplot(df_diff, x="pixel_dist", y="sympathy", hue="party",␣
↪hue_order=partiesVars, palette=party_cmap, order=1, scatter_kws={"s":1,␣
↪"alpha":0.3 }, y_jitter=0.015 )
plt.ylim(-0.05,1.05)
plt.xlim(-0.05,1.05)
```

Correlation between Sympathy and pixel distance of a voter: -0.5677017501802762

[91]: (-0.05, 1.05)

<Figure size 629.921x354.331 with 0 Axes>

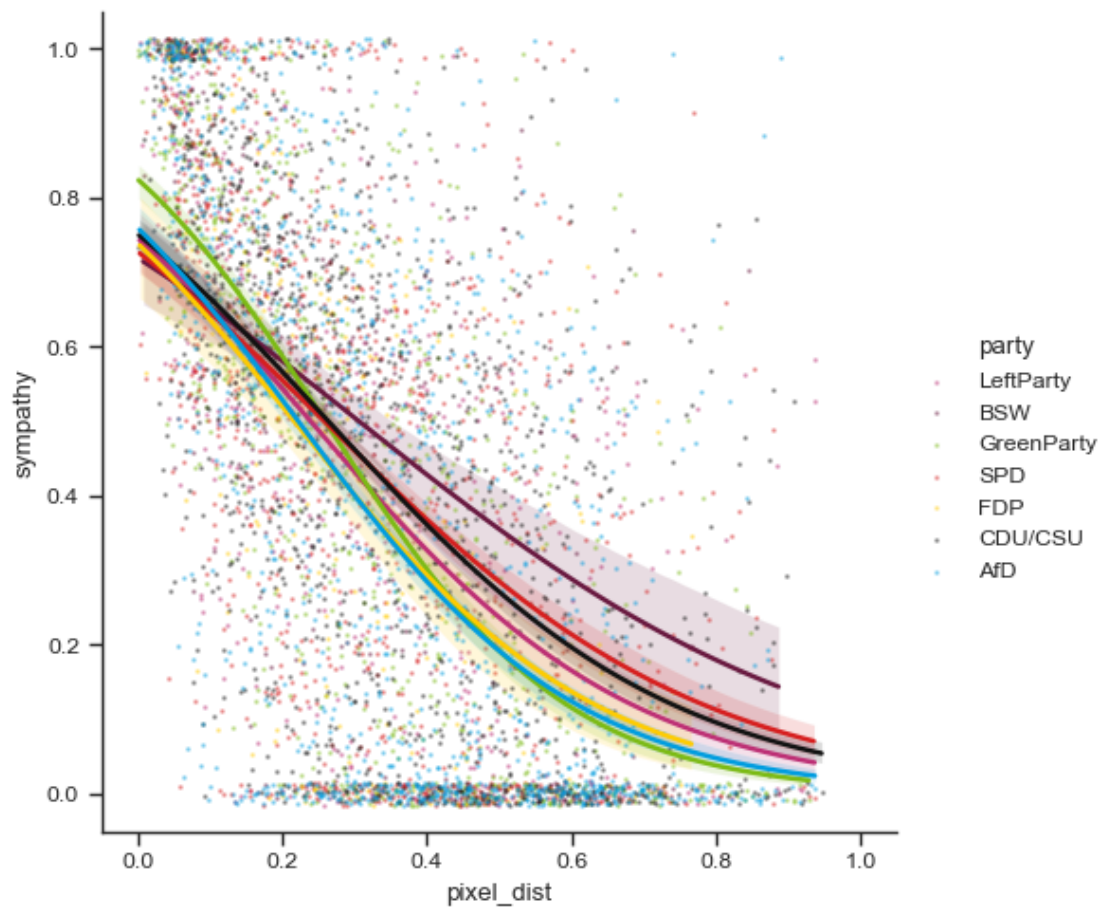


```
[93]: # %%
plt.figure()
print(f"Correlation between Sympathy and pixel distance of a voter:␣
      ↳{df_diff[['sympathy']+['pixel_dist']].corr().iloc[0,1]}")
sns.lmplot(df_diff, x="pixel_dist", y="sympathy", hue="party",␣
      ↳hue_order=partiesVars, palette=party_cmap, order=1, scatter_kws={"s":1,␣
      ↳"alpha":0.3 }, y_jitter=0.015, logistic=True )
plt.ylim(-0.05,1.05)
plt.xlim(-0.05,1.05)
```

Correlation between Sympathy and pixel distance of a voter: -0.5677017501802762

[93]: (-0.05, 1.05)

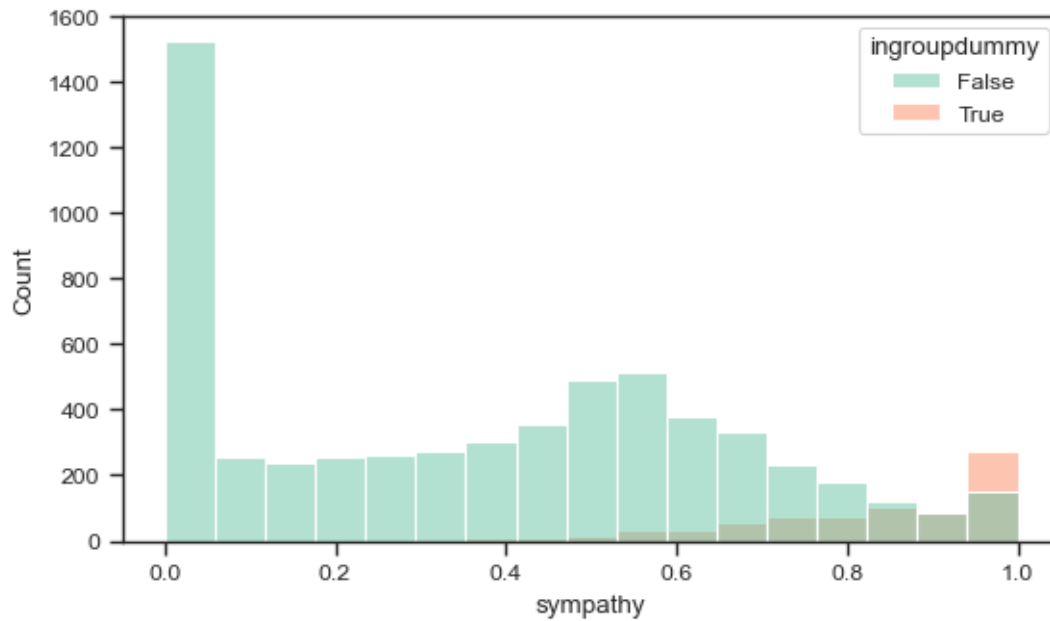
<Figure size 629.921x354.331 with 0 Axes>



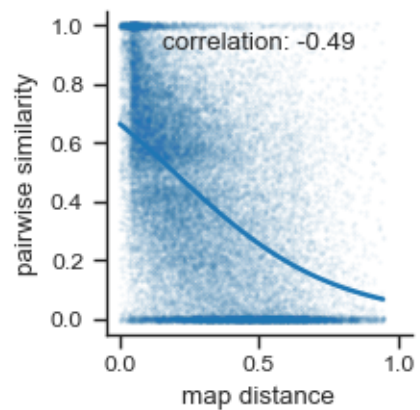
```
[94]: # %%

plt.figure()
sns.histplot(df_diff, x="sympathy", hue="ingroupdummy", palette="Set2",)
```

```
[94]: <Axes: xlabel='sympathy', ylabel='Count'>
```



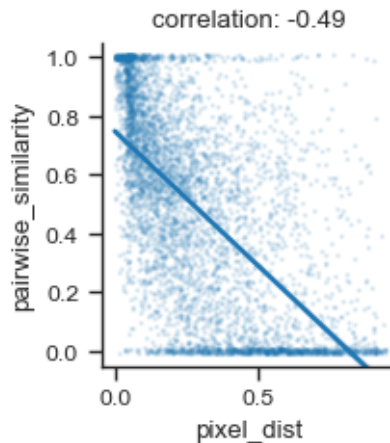
```
[95]: # %%
sns.lmplot(df_diff, x="pixel_dist", y="pairwise_similarity",
           scatter_kws={"marker": ".", "s": 0.1, "alpha": 0.1}, y_jitter=0.01, height=6/2.
           54, aspect=1, logistic=True,)
plt.ylim(-0.05, 1.05)
plt.xlim(-0.05, 1.05)
plt.title(f"correlation: {df_diff[['pairwise_similarity']+['pixel_dist']].
           corr().iloc[0,1]:.2f}", y=0.83)
plt.xlabel("map distance")
plt.ylabel("pairwise similarity")
plt.tight_layout()
plt.savefig("figs/correlation_pairwise_map.png", dpi=300)
```



```
[96]: # %%
# fig = plt.figure(figsize=(5/2.54,5/2.54))
print(f"Correlation between pairwise similarity and pixel distance:␣
↪{df_diff[['pairwise_similarity']+['pixel_dist']].corr().iloc[0,1]}")
sns.lmplot(df_diff.loc[df_diff.dot1=="self"], x="pixel_dist",␣
↪y="pairwise_similarity", scatter_kws={"marker":".", "s":1, "alpha":0.1 },␣
↪y_jitter=0.01, height=6/2.54, aspect=1)
plt.ylim(-0.05,1.05)
plt.title(f"correlation: {df_diff[['pairwise_similarity']+['pixel_dist']].
↪corr().iloc[0,1]:.2f}")
```

Correlation between pairwise similarity and pixel distance: -0.492941579912633

```
[96]: Text(0.5, 1.0, 'correlation: -0.49')
```



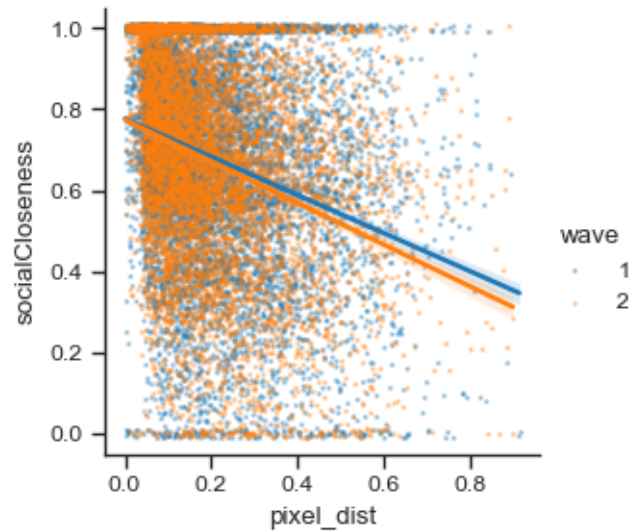
```
[100]: # %%

plt.figure(figsize=(16/2.54,9/2.54))
print(f"Correlation between social closeness and pixel distance:␣
↪{df_diff[['socialCloseness']+['pixel_dist']].corr().iloc[0,1]}")
df_diff["lr_cat"] =pd.cut(df_diff.lr, np.linspace(0, 1, 4), right=False,␣
↪labels=["left", "moderate", "right"])
sns.lmplot(df_diff, x="pixel_dist", y="socialCloseness", hue="wave",␣
↪scatter_kws={"s":1, "alpha":0.3 }, y_jitter=0.01, height=3)
plt.ylim(-0.05,1.05)
```

Correlation between social closeness and pixel distance: -0.28573363407348884

```
[100]: (-0.05, 1.05)
```

<Figure size 629.921x354.331 with 0 Axes>

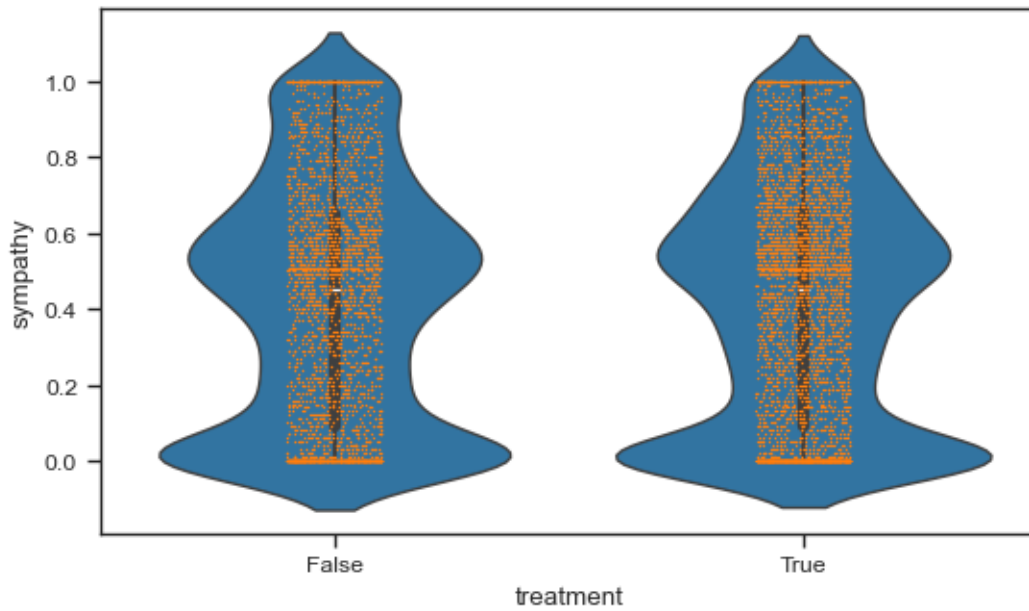


```
[ ]: # %%
```

```
[101]: # %%
df_diff["treatment"] = False
df_diff.loc[df_diff["id"].isin(df_p.loc[df_p.treatment_wave2==1, "id"].
    tolist()), "treatment"] = True

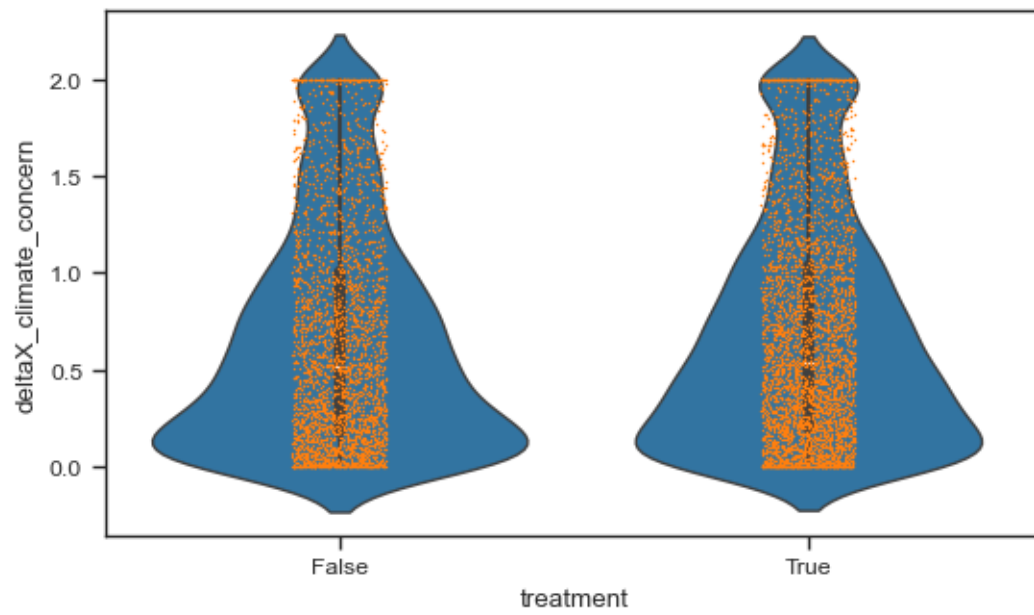
plt.figure()
sns.violinplot(df_diff.loc[df_diff.wave==2], x="treatment", y="sympathy")
sns.stripplot(df_diff.loc[df_diff.wave==2], x="treatment", y="sympathy",size=1)
```

```
[101]: <Axes: xlabel='treatment', ylabel='sympathy'>
```



```
[102]: # %%
plt.figure()
q = questions_sc[0]
sns.violinplot(df_diff.loc[(df_diff.wave==2) & (df_diff.dot2.isin(partiesVars)) &
    ↪ (df_diff.dot1=="self") ], x="treatment", y=f"deltaX_{q}")
sns.stripplot(df_diff.loc[(df_diff.wave==2) & (df_diff.dot2.isin(partiesVars)) &
    ↪ (df_diff.dot1=="self")], x="treatment", y=f"deltaX_{q}",size=1,)
```

```
[102]: <Axes: xlabel='treatment', ylabel='deltaX_climate_concern'>
```



[]: # %%

[]: # %%